

The Alberta Buck - Notes (DRAFT v0.2)

Private Bearer Instruments on an Identity-Aware Demurrage Currency

Perry Kundert

2026-04-18



For forty years cryptographers have tried to build digital cash and failed. Chaum's DigiCash (1990s) had the mathematics but went bankrupt for want of merchant adoption. Bitcoin gave us digital scarcity but no native bearer-paper form – Casascius coins and BIP-38 paper wallets are aftermarket wrappers, not first-class instruments. Central bank digital currencies threaten the opposite: digital surveillance with no bearer privacy at all.

The Alberta Buck identity, reserve, and demurrage machinery – combined with one new primitive, a global commitment pool – finally permits a native digital-private bearer instrument. This document specifies that primitive.

A **BUCK Note** is an on-chain commitment whose opening authorises a private mint to a registered identity. The same commitment can live only on chain (a software-only transfer between two of a user's own private accounts), travel between parties as a digital message, or be printed on paper as a QR-encoded slip. All three surfaces share the same cryptography and the same on-chain footprint.

A single Merkle tree of note commitments and a single nullifier set replace every parallel bearer-instrument architecture the Alberta Buck series has previously explored. Three *Identity bindings* parameterise the commitment:

- **Type 1 – Encrypted Issuer Identity.** Spend predicate requires the depositor's registered ElGamal credential to encrypt the same identity point the issuer's credential encrypts. A note Alice issues from one of her private accounts and later deposits at another: the two addresses share the underlying Identity scalar but are cryptographically unlinkable to any third party.
- **Type 2 – Public Issuer Identity.** Spend predicate requires the depositor to be any registered identity; the issuer's public identity is bound to the note in plaintext. A cheque whose maker is known, whose bearer is not.

- **Type 3 – Stealth-Targeted to Recipient.** Spend predicate requires the depositor to hold the secret of a stealth public key derived from the recipient’s registered identity. A cheque made out to a specific recipient, retrievable only by them, with no observable on-chain link from issuer to recipient.

Demurrage needs no new machinery. The Note Pool *is* simply the BUCK ERC-20 contract’s own balance – a regular BUCK account accumulating BUCK-age like any other account. Mint: the user transfers v BUCK to the contract (a standard ERC-20 transfer; sender pays their own outgoing demurrage), incrementing both the contract’s balance B and a scalar F (sum of face values of live notes). Redemption is an *age-preserving transfer* unique to the Notes contract’s single egress path: the recipient’s nominal balance rises by the full face value v , and their BUCK-age rises by $v \cdot A/B$, where A and B are the pool account’s accumulated BUCK-age and balance in the existing ERC-20 ledger. That is: the pool’s age-per-BUCK follows the redeemed value to the recipient, who then bears the pool’s share of demurrage on their *next* outgoing transfer via the standard BUCK accounting. This is exactly the cost a user would have paid had they held v BUCK personally through the pool’s residence period. Donations to the contract dilute A/B (donations add balance with no age), directly lowering the age-share absorbed by all subsequent redemptions. Conservation is automatic: balance and age are each exactly transferred, nothing is created or destroyed; the Jubilee continues to grow autonomously at $\kappa \cdot \text{totalSupply}$. No face discount, no division in the delivered amount, no F in the formula – only a single new primitive (age-preserving transfer), localized to one contract function.

A BUCK Note can be materialised as a physical paper slip with a QR-encoded opening, a hologram seal or scratch-off panel over the secret, and an issuer mark – indistinguishable in use from a \$20 bill while being cryptographically impossible to counterfeit. Demurrage solves physical cash’s oldest problem: a lost note’s value decays at the same rate as an on-chain balance, with the difference accruing to the Jubilee rather than as seigniorage to the issuer. Issuance is on chain and identity-bound; circulation is private; deposit is auditable. This is stronger compliance than physical cash provides today, and stronger privacy than any CBDC has ever proposed.

The consequence most relevant to users: only notes where the issuer cannot later race the holder to spend are safe for long-term holding. This is **Type 1** (issuer = depositor, same Identity, no third party knows the note secret) and **Type 3** (the issuer’s knowledge of the note secret does not suffice to spend – only the stealth-derived secret can). Type 2’s public-issuer attribution comes at the cost of retained issuer-spend capability, restricting it to short-lived use cases (cheques presented within hours or days, not stored for years). This is an intrinsic cryptographic property, not a policy choice. (PDF, Text)

Contents

1	The Cash Paradox	6
1.1	Why Forty Years of Trying	6
2	The Core Primitive: One Global Commitment Pool	8
2.1	The Note Contract State	8
2.2	The Note Commitment	8
2.3	The Note Nullifier	9
2.4	Lifecycle	9
3	The Surface of a Note: Chain, Message, Paper	10
3.1	Chain-Only Notes	10
3.2	Message Notes	10
3.3	Paper Notes	10
3.4	Counterfeiting is Cryptographically Impossible	11
3.5	Wallet-App Status Codes	11
4	Three Identity Bindings	12
4.1	Type 1: Encrypted Issuer Identity ("Wormhole")	12
4.2	Type 2: Public Issuer Identity ("Signed Cheque")	12
4.3	Type 3: Stealth-Targeted to Recipient ("Addressed Cheque")	13
4.4	Comparison of the Three Bindings	13
5	The Interaction Matrix: Note Binding x Account Mode	14
6	Demurrage at the Pool Level	16
6.1	Reminder: How BUCK Demurrage Actually Works	16
6.2	The Pool Account's Demurrage	16
6.3	The Age-Preserving Redemption	17
6.4	Donations Integrate Smoothly	17
6.5	Conservation is Automatic, and Tight	18
6.6	Numeric Example	18
6.7	Who Bears the Demurrage, Really	18
7	Demurrage Solves the Lost-Cash Problem	20
7.1	The Hidden Inflation of Lost Cash	20
7.2	What BUCK Demurrage Does For Lost Notes	20
7.3	The Symmetry With On-Chain Balances	21
8	Long-Term Holding Safety: The Printer-Trust Distinction	22
8.1	The Printer-Trust Model	22
8.2	Type 1 (Encrypted Issuer Identity): Safe	22
8.3	Type 2 (Public Issuer Identity): Unsafe for Long-Term Storage	22
8.4	Type 3 (Stealth-Targeted to Recipient): Safe	22
8.5	Summary: Which Notes Can Be Saved?	23

9	Physical Security Model	24
9.1	Layered Defense	24
9.2	Recommended Manufacturing Profile	24
10	Use Cases	26
10.1	Walking-Around Money	26
10.2	Cross-Border Remittances	26
10.3	Tip Jars and Charitable Donations	26
10.4	Cold Storage and Disaster Preparedness	27
10.5	Estate Planning	27
10.6	Gift Cards Without Issuer Risk	27
10.7	Tax-Compliant Compensation	27
10.8	Underbanked and Cash-Economy Inclusion	27
11	Selective Disclosure: Compliance Without Surveillance	29
11.1	The Asymmetry: Issuance vs. Circulation	29
11.2	The Regulatory Audit Path	29
11.3	What Compliance Looks Like	30
11.4	The CBDC Comparison	30
12	Comparison With Prior Art	31
12.1	Chaum's E-Cash (1982-1998)	31
12.2	Brands' Offline Divisible E-Cash (1993)	31
12.3	Casascius Coins (2011-2013)	31
12.4	BIP-38 Paper Wallets	32
12.5	Zerocash / Zcash (2014-)	32
12.6	Aztec / Zk.Money	32
12.7	Tornado Cash and Privacy Pools	33
12.8	Comparison Summary	33
12.9	Comparison: Cash, CBDC, BUCK Notes	33
13	Required Identity Cryptography	35
13.1	Components Already Present	35
13.2	New Primitive: The Note Commitment Hash	35
13.3	New Primitive: The Nullifier PRF	35
13.4	Re-Used Primitive: Theorem 7 (Stealth Address)	36
13.5	The Re-Encryption Operation at Mint	36
13.6	The Re-Encryption Operation at Spend (Type 1 Only)	36
13.7	The Schnorr PoK Operation at Spend (Type 2 and Type 3)	36
13.8	Summary: New Cryptographic Components	36
14	The SNARK Circuits	37
14.1	Mint Circuit	37
14.2	Spend Circuit (Type 1 – Encrypted Issuer)	37
14.3	Spend Circuit (Type 2 – Public Issuer)	38
14.4	Spend Circuit (Type 3 – Stealth-Targeted)	38
14.5	Circuit Engineering Budget	39
14.6	Common Circuit Gadgets	39

15 What This Architecture Rules Out: Pure-Bearer Cryptographic Notes	40
16 Security Assumptions and Theorems Used	41
16.1 Demurrage Conservation as a Contract Invariant	41
17 Implementation Scope	42
17.1 Smart Contract Additions	42
17.2 SNARK Circuits	42
17.3 Wallet & UX	42
17.4 Cross-Checks and Audit	43
17.5 Phased Rollout	43
17.6 Regulatory and Legal Engagement	43
18 Open Questions	44
18.1 Donation Incidence	44
18.2 Stranded Value on Lost Notes	44
18.3 Integer-Arithmetic Rounding	44
18.4 Predicate Programmability vs. Circuit Simplicity	44
18.5 Universal Spend Circuit vs. Three	44
18.6 Regulatory Interaction with the Age-Preserving Transfer	45
19 Summary	46

DRAFT

1 The Cash Paradox

Physical cash is simultaneously irreplaceable and impossible to digitize.

It is irreplaceable because it does things no digital instrument has ever matched. A \$20 bill works without a network, a battery, a bank, an ID check, or a counterparty to your transaction. It clears instantly, finally, and without fees. It is the single most successful monetary instrument in human history – so successful that roughly half the value of US currency in circulation is held overseas as a global store of value, in defiance of every theoretical reason it shouldn't be.¹

It is impossible to digitize because every attempt has reproduced the surveillance properties of bank money instead of the privacy properties of cash. Modern card networks log every transaction. Bank wire transfers require both parties to be identified. Stablecoins record every transfer publicly on chain. Central bank digital currencies, where they exist or are being designed (China's e-CNY, Europe's digital euro proposals), are explicitly programmable surveillance tools – the central bank can see, restrict, expire, or geofence any holder's balance at will.²

The closest existing analogues to digital cash are aftermarket bearer wrappers around cryptocurrencies: Casascius physical Bitcoin coins (2011-2013), OpenDime USB sticks, BIP-38 paper wallets, and the various scratch-card "crypto gift cards" sold at convenience stores. These work, technically – a Casascius coin is a genuine bearer instrument – but they are workarounds, not designs. The underlying token (Bitcoin) does not natively understand bearer-paper form. Loss of the physical coin is permanent loss of the value. Counterfeiting requires faith in the manufacturer's hologram, not in the cryptography.³

What if the bearer-paper form were a *first-class* instrument of the system? Designed in from the start, with the same cryptographic and economic properties as on-chain balances, integrated with the identity and reserve mechanics that already exist?

That is what BUCK Notes is.

1.1 Why Forty Years of Trying

David Chaum invented digital cash in 1982 with the blind-signature e-cash scheme.⁴ By 1989 he had founded DigiCash to commercialize it. By 1998 DigiCash was bankrupt – not because the cryptography failed (it worked), but because no merchant wanted to be the first to accept it and no consumer wanted to be the first to hold it. The two-sided market never bootstrapped.

Stefan Brands extended Chaum's work in 1993 with offline divisible coins that revealed the spender's identity if (and only if) they double-spent.⁵ Camenisch, Hohenberger, and Lysyanskaya improved on this in 2005 with compact divisible e-cash.⁶ None of these schemes ever shipped in

¹Federal Reserve. "Currency in Circulation" (2024). Approximately half of US currency in circulation by value is held outside the United States, providing a global store-of-value function despite zero yield.

²Bank for International Settlements. "Rise of the Central Bank Digital Currencies" (2020). Surveys CBDC designs across major central banks; notes that nearly all incorporate explicit issuer-side surveillance and programmability features (spending limits, geofencing, expiration, freezing) that physical cash does not have.

³Caldwell, M. "Casascius Physical Bitcoins" (2011-2013). Physical brass and silver coins containing Bitcoin private keys under tamper-evident holographic seals. Production halted in 2013 after FinCEN classified the issuer as a money transmitter requiring federal licensing. Surviving "unpeeled" Casascius coins now trade at substantial premiums above their Bitcoin face value as collectibles.

⁴Chaum, D. "Blind Signatures for Untraceable Payments" (1982). *Advances in Cryptology - Crypto '82*, pp. 199-203. The foundational paper introducing blind signatures and digital cash. Chaum's DigiCash company commercialized the design from 1989-1998 before bankruptcy.

⁵Brands, S. "Untraceable Off-line Cash in Wallets with Observers" (1993). *Advances in Cryptology - Crypto '93*, pp. 302-318. Offline divisible e-cash with double-spend identity revelation – a deterrent rather than prevention model.

⁶Camenisch, J., Hohenberger, S., Lysyanskaya, A. "Compact E-Cash" (2005). *Advances in Cryptology - Eurocrypt*

production at scale.

Bitcoin, when it arrived in 2009, was deliberately *not* digital cash in the Chaumian sense. It was digital scarcity with public ledger transparency – closer to digital gold than digital cash. The bearer-paper market that grew up around it (Casascius, OpenDime, paper wallets) was an afterthought, not a design goal.

Zerocash (2014)⁷ and its production deployment as Zcash gave us the first scalable on-chain commitment-and-nullifier UTXO model – the cryptographic primitive that BUCK Notes use. Zcash showed that you can have private on-chain balances; it did not extend that to printable physical bearer instruments. Aztec⁸ extended the model with predicate-encrypted notes for private smart contract calls, but again, no paper.

Tornado Cash⁹ reduced the primitive to its simplest form (fixed-denomination commitments in, anonymous withdrawals out) and was OFAC-sanctioned in 2022. Privacy Pools¹⁰ extended Tornado with per-spend association-set proofs, restoring compliance paths without abandoning the privacy primitive.

The reason all of this never produced BUCK Notes – a printable, demurrage-aware, identity-aware private bearer instrument – is that it required four pieces to come together simultaneously, and they did not until now:

1. A **commitment-and-nullifier UTXO model** on a smart contract platform (Zerocash gave us this in 2014; Aztec deployed it on Ethereum in 2018-2020).
2. A **cryptographic identity layer** that lets some notes be targeted to a specific recipient and others be self-wormholed between a user's own unlinkable accounts (the BUCK identity system, with PS signatures, ElGamal credentials, and Chaum-Pedersen re-encryption – see Identity).
3. A **demurrage mechanism** that solves the lost-cash problem and gives bearer notes a finite lifespan (BUCK's demurrage and Jubilee are unique among monetary systems).
4. A **jurisdictional reserve** that backs the value, so the bearer note represents a real claim on real assets, not a speculative crypto-token whose price might collapse between print and deposit (Alberta Buck's commodity-basket-stabilized reserve and the BuckK PID controller).

Alberta Buck has all four. No prior system had all four. That is why BUCK Notes is a *design*, not a workaround.

⁷Ben-Sasson, E., Chiesa, A., Garman, C., Green, M., Miers, I., Tromer, E., Virza, M. "Zerocash: Decentralized Anonymous Payments from Bitcoin" (2014). IEEE S&P 2014. The foundational design for on-chain commitment-and-nullifier UTXO privacy, deployed in production as Zcash since 2016 – the commitment-and-nullifier primitive used by BUCK Notes.

⁸Williamson, Z., et al. "Aztec Protocol" (2018-). Ethereum L2 rollup providing general-purpose private smart contracts using PLONK proofs and predicate-encrypted notes. The cryptographic ancestor of the **predicate** field in BUCK Note commitments.

⁹Pertsev, A., Semenov, R., Storm, R. "Tornado Cash Privacy Solution" (2019). Fixed-denomination commitment-and-nullifier mixer on Ethereum. Sanctioned by the US Office of Foreign Assets Control (OFAC) in August 2022 for alleged use in money laundering by North Korean state actors.

¹⁰Buterin, V., Schaefer, J., Tromer, E., et al. "Blockchain Privacy and Regulatory Compliance: Towards a Practical Equilibrium" (2023). Extends commitment-pool mixers with per-spend "association set" proofs, allowing depositors to demonstrate non-membership in incriminating subsets without revealing their specific deposit.

2 The Core Primitive: One Global Commitment Pool

The entire BUCK Notes design rests on a single on-chain primitive: a Merkle tree of commitments with a nullifier set, accompanied by an age-preserving transfer that delivers the opening’s face value plus the pool’s proportional BUCK-age to a recipient. Everything else – paper notes, targeted cheques, self-wormholes, stealth deposits – is a consequence of varying the *spend predicate* inside the commitment.

2.1 The Note Contract State

The BUCK contract gains exactly three new pieces of state:

State	Type	Purpose
noteTree	Merkle tree of commitments	Cryptographic anchor for every outstanding note
nullifiers	Set of used nullifiers	Double-spend prevention
noteFaceSum	F in $\mathbb{Z}$	Sum of face values of all outstanding notes

That is all. The pool’s nominal BUCK balance B and accumulated BUCK-age A are not tracked as separate state: *the pool is just a BUCK account*, specifically the ERC-20 contract’s own address, and B, A are exactly the balance and BUCK-age the ERC-20 ledger already maintains for it. No lazy integration, no shadow aggregates, no separate **poolT** timestamp. **noteFaceSum** is a pure audit scalar – the total face-value liability outstanding – and does not appear in the redemption formula itself.

BUCK transferred to the contract address (via mint, donation, or accident) joins B through the normal ERC-20 path. BUCK transferred out of the contract on redemption uses a *single new primitive*: an age-preserving transfer that moves both balance and proportional BUCK-age from the contract to the recipient. This is the only non-standard BUCK operation in the design.

2.2 The Note Commitment

A note commitment is a Poseidon hash of the fields the SNARK will later open:

$$cm = H_{cm}(\text{type}, v, \rho, \text{id}_{\text{payload}}, \text{predicate})$$

with:

- **type** $\in \{1, 2, 3\}$: the Identity-binding mode
- **v** $\in \mathbb{Z}_q$: face value
- **ρ** : per-note randomness (source of the nullifier)
- **id_{payload}**: Identity-binding data (see next section)
- **predicate**: optional additional spend predicate (time locks, oracle conditions, multi-sig; not elaborated in detail here – a single bit **predicate** = None= keeps the SNARK to the three base types. See "Open Questions" for bounded-programmability notes.)

No per-note timestamp or baked-in BUCK-age: demurrage is computed at redemption time from the *pool’s* account state (B, A) , not from the individual note’s life history, so no per-note age fields need to ride in the commitment.

The commitment is inserted as a leaf of **noteTree** at mint time.

2.3 The Note Nullifier

The nullifier is a pseudo-random function of the note’s secret witness material:

$$nf = \text{PRF}_{\text{key}}(\rho)$$

where `key` is the spend-side secret specific to the note type (the issuer’s sk in Type 1, any registered spender’s sk in Type 2, the stealth sk^* in Type 3). The PRF is instantiated via Poseidon in the SNARK-friendly setting.

The nullifier is emitted on deposit; double-spend is rejected by set-membership check.

2.4 Lifecycle

`mint (burn):`

```
# Standard ERC-20 transfer from caller to contract address.
# Caller pays their own outgoing-transfer demurrage through the
# normal BUCK accounting path. Contract balance B gains v; its
# accumulated BUCK-age A starts earning on the new v from here.
transfer v BUCK from caller to contract
for each of {cm_1, ..., cm_N}:
    append cm_i to noteTree
noteFaceSum += sum(v_i)
```

`deposit (mint-out):`

```
verify SNARK:
- Merkle inclusion of some cm in noteTree
- Knowledge of (v, rho, id_payload, predicate)
- Predicate satisfied by witness (type-specific)
- nullifier correctness
- v is a public output of the SNARK
require nf not in nullifiers
nullifiers += nf
# Read pool state from the ERC-20 ledger:
(B, A) = (balanceOf(contract), buckAgeOf(contract))
ageShare = v * A / B
# Age-preserving transfer (the one new primitive):
# contract.balance -= v; contract.age -= ageShare
# recipient.balance += v; recipient.age += ageShare
transferPreservingAge(contract, recipient, v, ageShare)
noteFaceSum -= v
```

Full face value is delivered: the recipient’s nominal balance goes up by exactly v . The pool’s demurrage cost rides along as the BUCK-age $v \cdot A/B$ added to the recipient’s account. The recipient will bear that demurrage on their *next* outgoing transfer through the ordinary BUCK accounting path – the same path any holder travels, but on an age they acquired by accepting the note. No Jubilee payment is emitted from inside the deposit; the Jubilee grows autonomously at $\kappa \cdot \text{totalSupply}$, which continues to include both B (before redemption) and v (after, as part of recipient’s balance).

3 The Surface of a Note: Chain, Message, Paper

A BUCK Note is, fundamentally, a tuple

$(\text{type}, v, \rho, \text{id_payload}, \text{predicate}, \text{merkle_path})$

that opens to a commitment $cm = H_{\text{cm}}(\text{type}, v, \rho, \text{id_payload}, \text{predicate})$ in the on-chain note tree. The tuple can live on any medium capable of storing ~200-500 bytes and being read back faithfully. Three surfaces are supported:

3.1 Chain-Only Notes

The opening is transmitted privately from issuer to depositor (for Type 1, both are the same user using different accounts) through an out-of-band encrypted channel – typically the wallet’s internal storage, or a direct peer-to-peer message between wallets. The note never leaves the issuer’s control infrastructure. This is the Type 1 "self-wormhole" use case: Alice mints a Type 1 note from a private account, hands the opening to herself, and deposits into a second unlinkable account.

3.2 Message Notes

The opening travels as a digital message: an email, SMS, QR shown on a screen, signal-messenger attachment, NFC handoff. No physical artifact exists, but the recipient can store, forward, or print the opening at their convenience. Type 2 and Type 3 notes used between parties who have reliable digital communication occupy this surface.

3.3 Paper Notes

The opening is printed as a QR code on a slip of paper, a card, a steel plate, or a sticker. For Type 1 and Type 3 notes (where the cryptography already prevents a photographing observer from spending), no physical seal is required. For Type 2 notes (where whoever reads ρ can spend), the QR must be protected by a tamper-evident seal – hologram, scratch-off panel, or laminated sleeve – so the holder can verify they were the first to expose the secret.

A printed note typically also carries:

- A visible denomination (1, 5, 10, 100, 1,000, 10,000 BUCK) for human convenience. The actual value is what the commitment encodes; the printed face is metadata that a wallet verifies before deposit.
- A human-readable issuer mark, for visual recognition. Cryptographic authenticity comes from the on-chain commitment, not the mark.
- Redundant QRs front and back with Reed-Solomon error correction, plus the secret in base32 digits for manual recovery.

Paper notes are the form most people will think of when they hear "BUCK Note." Physical paper is the surface of last resort: it works offline, without batteries, across borders, through power outages, and past the expiration of any given digital communication channel.

3.4 Counterfeiting is Cryptographically Impossible

A "fake" BUCK Note is any opening whose claimed commitment does not appear in the on-chain note tree. The Merkle inclusion proof inside the spend SNARK fails. No mint occurs. No second-guessing is required: there is no paper-inspection threshold to deceive and no "close enough" forgery that a harried cashier might accept.

This is a stronger anti-counterfeit guarantee than the Bank of Canada provides for physical \$20 bills. Polymer banknotes can be counterfeited (the techniques are well-published; the question is only whether the counterfeit passes casual inspection). A BUCK Note cannot be counterfeited at all – the on-chain commitment registry is the authentic-bill list, and any printed note whose commitment is not in the tree will produce a **FAKE** wallet status on scan and nothing more.

3.5 Wallet-App Status Codes

A user-facing wallet that scans a BUCK Note (from chain storage, message, or paper QR) displays one of five statuses:

Display	Meaning
VALID: 1,000 BUCK	Commitment in tree, nullifier unspent, spend predicate satisfiable
DEAD	Commitment in tree, nullifier already spent
FAKE	Commitment not in tree
EXPIRED	Predicate includes an expiry that has passed
NOT FOR YOU	Type 3 (or Type 1 with a different Identity) and the viewer is not the intended spender

Casual users do not need to understand the underlying cryptography any more than they need to understand the security features of a \$20 bill. They scan, see **VALID**, and accept the note as payment.

On Deposit, the wallet constructs the SNARK locally (or via a delegated prover service for thin clients), submits the transaction, and displays the result. SNARK construction for a single-note deposit is on the order of seconds on a modern smartphone – Aztec’s Noir compiler and Zcash’s Halo2 prover both run on mobile.

After successful deposit, the note’s nullifier is consumed and the QR is permanently dead. A torn-in-half paper note is a *human* signal of finality, not a cryptographic one – but it’s a useful ritual to prevent confused future holders from trying to redeposit a dead QR.

4 Three Identity Bindings

Every note commits to one of three *Identity-binding* payloads. The binding determines who can spend the note and what observers learn about the issuer.

4.1 Type 1: Encrypted Issuer Identity ("Wormhole")

Identity payload.

$$\text{id}_{\text{payload}}^{(1)} = E_{\text{issuer}} = (R_i, C_i) = (r_i G, M_{\text{issuer}} + r_i \cdot pk_{\text{issuer}})$$

where pk_{issuer} is the issuer's own identity public key at mint time, $M_{\text{issuer}} = m_{\text{issuer}} \cdot G$, and r_i is fresh randomness. This is a standard ElGamal encryption of the issuer's Identity point under their own key – the same structure registered credentials use.

Spend predicate. A depositor with registered credential $E_{\text{dep}} = (R_d, C_d)$ under their own pk_{dep} submits a SNARK proving the Chaum-Pedersen relation $C_i - sk_{\text{issuer}} R_i = C_d - sk_{\text{dep}} R_d$. That is, the two credentials decrypt to the same Identity point.

This is Theorem 6 of the proofs document (Proofs) lifted verbatim into the SNARK; a SNARK-verified instance of the CP relation implies $m_{\text{issuer}} = m_{\text{dep}}$, i.e., *same underlying Identity*.

Consequence. Only accounts whose underlying Identity scalar matches the issuer's can spend a Type 1 note. This is the Wormhole semantics: Alice mints a Type 1 note from address A_1 and later deposits at address A_2 , where both addresses carry Alice's Identity (different registered credentials, same M). Observers cannot link the two addresses beyond the Identity-constant relation they already have.

4.2 Type 2: Public Issuer Identity ("Signed Cheque")

Identity payload.

$$\text{id}_{\text{payload}}^{(2)} = (m_{\text{issuer}}, \sigma_{\text{issuer}})$$

where $m_{\text{issuer}} \in \mathbb{Z}_q$ is the issuer's Identity scalar *in plaintext* and σ_{issuer} is a Schnorr signature on cm under the issuer's registered $pk_{\text{issuer}} = m_{\text{issuer}} \cdot G$ (or a signed commitment to the note; details equivalent).

Anyone reading the commitment openings learns who issued the note: the Identity point $M_{\text{issuer}} = m_{\text{issuer}} \cdot G$ is in the clear, and since this is a public-identity account, its real-world identity is world-readable from the IdentityRegistry.

Spend predicate. The SNARK proves:

1. Knowledge of ρ such that $nf = \text{PRF}_\rho(\cdot)$
2. σ_{issuer} verifies against the registered pk_{issuer} in IdentityRegistry (SNARK-inside signature verification)
3. The depositor is *some* registered identity: Schnorr proof of knowledge of sk_{dep} for a pk_{dep} that IdentityRegistry attests to (anonymous-valid) or whose public identity is published (public)

No CP-equality is required – the issuer's identity is already known, the depositor's identity is anyone properly registered.

Consequence. Any registered identity can deposit. The issuer retains the right to spend their own note (by virtue of knowing ρ), so holders of Type 2 notes bear *printer-race risk*: if the issuer kept ρ , they can race the holder to deposit. Type 2 is appropriate for transactions that close within hours or days, not long-term holding. See section "Long-Term Holding Safety".

4.3 Type 3: Stealth-Targeted to Recipient ("Addressed Cheque")

Identity payload.

$$\text{id}_{\text{payload}}^{(3)} = (R_e, pk^*)$$

where $R_e = r_e G$ is an ephemeral published with the note and

$$pk^* = H_{\text{st}}(r_e \cdot pk_{\text{recipient}}) \cdot pk_{\text{recipient}}.$$

$pk_{\text{recipient}}$ is *not* in the payload – it would reveal the target to observers. Only the recipient, scanning notes with their own $sk_{\text{recipient}}$, can recognise a note as addressed to them: for each candidate note they compute $s = sk_{\text{recipient}} \cdot R_e$ and check whether $H_{\text{st}}(s) \cdot pk_{\text{recipient}}$ equals the note's pk^* (see Corollary 3 of Theorem 7 in Proofs).

Spend predicate. The SNARK proves knowledge of a scalar α with $\alpha \cdot G = pk^*$. By Theorem 7 this implies the prover knows the stealth secret $sk^* = H_{\text{st}}(s) \cdot sk_{\text{recipient}}$, which reduces to knowing $sk_{\text{recipient}}$ (the issuer cannot compute sk^* from $r_e, pk_{\text{recipient}}$ alone without breaking DLog).

The depositor also produces a Schnorr PoK of knowledge of sk_{dep} for their own registered pk_{dep} , binding the deposit to a compliant mint-out identity. Optionally, the recipient's credential $E_{\text{recipient}}$ can be used directly: the SNARK proves CP-equality between the decrypted $M_{\text{recipient}}$ and the depositor's registered E_{dep} , making the deposit identity-binding. The minimal-privacy variant omits this and lets any registered depositor submit the stealth-spend (suitable when the recipient wants pseudonymous redemption to a fresh account).

Consequence. Only the recipient – the holder of $sk_{\text{recipient}}$ – can spend the note. The issuer *cannot* spend what she printed (Theorem 7, Corollary 1 – "printer non-fungibility"). Third parties who see the published note learn nothing about $pk_{\text{recipient}}$. Type 3 is safe for long-term holding because the issuer's knowledge of r_e does not translate into spending power.

4.4 Comparison of the Three Bindings

Property	Type 1 Encrypted Issuer	Type 2 Public Issuer	Type 3 Stealth-Targeted
Issuer known to observers	No (ElGamal-encrypted)	Yes (plaintext)	No (not in payload)
Recipient known to observers	No (anyone with same M can spend)	No (any registered identity can spend)	No (stealth addr)
Observer links burn-mint	Only via timing; not by identity	Public flow with anonymous recipient	Not at all
Issuer can spend own note	Yes (Wormhole self-case)	Yes (knows ρ)	No (Theorem 7 Corollary 1)
Long-term holding safe	Yes (issuer = intended spender)	No (issuer race)	Yes (printer non-fungibility)
Primary use	Private self-transfer; custody moves	Cashier's cheques; payroll; donations	Addressed remittances; gifts
Spend-circuit soundness	Theorem 6 (CP-equality)	Schnorr + signature-in-SNARK	Theorem 7 (stealth Schnorr)

The three types together span every use case that a private bearer-instrument layer on an identity-aware currency can support. A fourth option – pure bearer notes with no identity hook at all – is intentionally absent; see "What This Architecture Rules Out" below.

5 The Interaction Matrix: Note Binding x Account Mode

The current Identity system (Identity) distinguishes two registered-account visibility modes:

- **Anonymous-valid.** The account is provably KYC-verified but the underlying Identity is encrypted. Counterparties need a bilateral **approve** handshake to learn who they are dealing with.
- **Public.** The account publishes its Identity claim in the clear. Counterparties see the real-world Identity without an **approve** step.

Permissionless accounts are excluded: they have no registered Identity and therefore cannot be either the minting side of a note nor the depositing side. Every cell below requires at least one endpoint to be a registered identity.

Crossing the three note bindings with the two account modes at deposit time produces six cells. Each row corresponds to a note *type*; each column corresponds to the *depositor account* mode.

Note type	Anonymous-valid depositor	Public depositor
1. Encrypted issuer	Wormhole, private-private Alice (issuer) and Alice' (depositor) share M ; both anonymous-valid; no link. Canonical Wormhole.	Wormhole, private-to-public Alice (issuer, private) deposits to her public account. Observers see: the public account received value from some pool note. Use: moving wealth from private trading to a public identity.
2. Public issuer	Public-to-private cheque Issuer X (public) writes a cheque; any registered anonymous-valid account cashes it. Recipient identity hidden from public. Use: payroll to private employees; charitable disbursement; B2C payments.	Public-to-public transfer by cheque Both parties public. Equivalent to a plain public transfer except the note interposes a delay/indirection. Useful: pre-signed bearer cheques, printed and mailed before recipient is determined.
3. Stealth-targeted	Private-to-private addressed cheque Issuer (any) targets a specific anonymous-valid recipient. Only recipient can spend. No public link between issuer and recipient. Use: targeted remittances, inheritance, estate transfers.	Private-to-public addressed cheque Issuer targets a public identity. The public identity is still not in the payload (stealth addr); only the recipient recognizes the note on scan. Use: donations to a public charity from private accounts without a traceable transfer.

Several observations:

1. **The Wormhole's full functionality (Type 1, Cell 1 and 2)** collapses to a single note-pool spend circuit. No burn addresses, no `_mint_noincrease`, no approval machinery for pseudo-accounts.
2. **Public-issuer cheques (Type 2)** supply the long-missing "cashier's cheque" primitive: a public issuer commits to a transfer that any registered holder can realise. This is the natural form for payroll issuance, charitable disbursements, and attribution-preserving payments that current BUCK flow lacks.
3. **Stealth-targeted cheques (Type 3)** provide the "personal cheque" semantics – addressed to a specific recipient – with the property that the depositor can be a distinct *account* from their registered identity if desired (the SNARK binds the stealth secret, not the specific deposit address).
4. **Public-to-public transfers via Type 2 (Cell 2.2)** are a degenerate case: the same information flow could happen via a direct public **transfer**. The value of Cell 2.2 is the *timing*: the cheque can be printed, mailed, or held by the issuer before the recipient decides which of their public accounts to deposit to.

5. **Account modes are independent of issuance:** the issuer’s account visibility at mint time and the depositor’s account visibility at deposit time are chosen independently. The matrix above shows the *depositor* dimension; a full 3x2x2 enumeration would duplicate most cells without adding structural insight.

DRAFT

6 Demurrage at the Pool Level

The central simplification of this design: **the Note Pool is just a regular BUCK account.** Specifically, it is the BUCK ERC-20 contract's own balance. BUCKs transferred to the contract's address join the pool; BUCKs transferred out (as note redemptions) leave the pool. The contract account accumulates BUCK-age like every other account and participates in the ordinary demurrage accounting with no special-case code. The only piece of additional state is a single scalar F , the sum of face values of live notes.

6.1 Reminder: How BUCK Demurrage Actually Works

Demurrage is *not transferred* on any per-account basis. It is an accounting entry maintained on each account's state:

- Every block, the Jubilee account's balance grows by $\kappa \cdot \text{totalSupply} \cdot \Delta t$. This is a single global operation; the sum of all other accounts' nominal balances is held constant.
- Each account X has a running BUCK-age A_X that grows at rate B_X (its nominal balance) per unit time.
- When account X transfers v BUCK out, its spendable fraction is $(B_X - \kappa A_X)/B_X$; the outgoing transfer costs the account $v \cdot B_X / (B_X - \kappa A_X)$ of its nominal balance, and the corresponding $\kappa A_X \cdot v / (B_X - \kappa A_X)$ demurrage "owed" is simply extinguished from (B_X, A_X) against the Jubilee's autonomous growth.

System-wide invariant (preserved by every transfer):

$$\sum_{\text{all accounts } X} B_X + B_{\text{Jubilee}} = \text{constant}.$$

The Jubilee balance reflects the aggregate accrual $\kappa \int_0^t \text{totalSupply}(s) ds$; individual accounts' κA_X quantities track *their share* of that total.

6.2 The Pool Account's Demurrage

Treat the BUCK contract address as an ordinary account. Let:

- B = the contract's ERC-20 balance (= sum of all BUCK transferred in via mints and donations, minus all BUCK transferred out via redemptions, modulo the standard demurrage adjustment on outgoing transfers).
- A = the contract's accumulated BUCK-age, as maintained by the ERC-20 ledger.

B and A are already on chain; the Notes design queries them, it does not compute them. The pool's *effective spendable value* at any moment is

$$E = B - \kappa \cdot A.$$

This is the amount the contract would be able to transfer out if it emptied itself in a single call – exactly the same definition that applies to any other account.

6.3 The Age-Preserving Redemption

When a note of face v is redeemed, the contract performs an *age-preserving transfer* of v BUCK from itself to the recipient:

Side	Balance change	BUCK-age change
Pool	$-v$	$-v \cdot A/B$
Recipient	$+v$	$+v \cdot A/B$

Full face value is delivered; the pool’s BUCK-age *proportional share* – one average age-per-BUCK times face v – moves along with the balance to the recipient. The recipient’s account absorbs that age just as if they had personally held v BUCK through the pool’s residence period.

The pool’s *own* age-per-BUCK ratio A/B is preserved across the redemption: $\backslash (A_{\text{new}}/B_{\text{new}}) = (A - vA/B) / (B - v)$

- $v) = A(B - v) / (B(B - v)) = A/B \backslash$. The ratio drifts only

through time (age accrues at rate B) and through mints (fresh BUCK arrives age-free, diluting the ratio).

Contrast with a standard BUCK transfer, where the sender bears the demurrage on their side (balance reduced by $v \cdot B/(B - \kappa A)$) and the recipient receives a clean v with no age. The age-preserving variant simply *follows* the balance with its proportional age, deferring the demurrage reckoning to the recipient’s next outgoing transfer. This is an economically natural choice for a Note recipient: they have accepted the note in lieu of a direct BUCK holding, and in return accept the demurrage cost that goes with having held that BUCK.

6.4 Donations Integrate Smoothly

Any BUCK transferred to the contract address *without* minting a corresponding note increases B without adding BUCK-age (the sender’s age stays with the sender per standard ERC-20 semantics). The pool’s ratio A/B drops; subsequent redemptions carry a *smaller* age-share per face unit. Over a sequence of redemptions the donation is consumed as a demurrage subsidy for live noteholders.

Four origin cases, all handled identically:

1. **Accidental transfers.** A user fat-fingers the contract as a recipient. Their BUCK is not burned; it dilutes A/B , and future redeemers bear marginally less age.
2. **Philanthropic donations.** A user deliberately sends BUCK to the contract to subsidise noteholders. The subsidy spreads across all future redemptions until A/B drifts back up.
3. **Recovered collateral.** Mechanism-level refunds (e.g., an issuer’s collateral unlock) sent to the contract address.
4. **Rounding / dust.** Integer-arithmetic residuals. Self-healing.

After all notes are eventually redeemed, any remaining donation principal sits at $F = 0, B = D_{\text{residual}}, A = 0$ – unclaimable by redemption, but available to subsidise a fresh round of mints if the pool is used again.

6.5 Conservation is Automatic, and Tight

The age-preserving transfer conserves both quantities *exactly*:

- $\Delta B_{\text{pool}} + \Delta B_{\text{recipient}} = 0$.
- $\Delta A_{\text{pool}} + \Delta A_{\text{recipient}} = 0$.

Total BUCK in circulation is unchanged by redemption; total accumulated BUCK-age is unchanged by redemption. The Jubilee continues to grow autonomously at rate $\kappa \cdot \text{totalSupply}$ per block; this total is unaffected by whether the v BUCK sits in the pool or in the recipient's account. And whether the demurrage on that age is eventually paid out by the pool's (hypothetical) transfer or by the recipient's real outgoing transfer, the same $\kappa \cdot v \cdot A/B$ amount of spendable value will be reconciled against the Jubilee.

No pool-specific conservation theorem is needed beyond the system-wide $\sum_X B_X + B_{\text{Jubilee}} = \text{const}$ (automatic) and the local observation that the age-preserving transfer is two-sided balanced (evident from its definition).

6.6 Numeric Example

Suppose the pool holds $B = 10\,000\,000$ BUCK with $A = 5\,000\,000$ BUCK-years (average age 0.5 yr), and $F = 10\,000\,000$ BUCK of live notes outstanding.

Alice redeems a note of face $v = 1,000$:

- Alice's balance rises by 1,000 BUCK.
- Alice's BUCK-age rises by $1,000 \cdot 5,000,000 / 10,000,000 = 500$ BUCK-years. In nominal units: her 1,000 incoming BUCK has an effective *age* of 0.5 yr attached to it.
- When Alice next spends 1,000 BUCK from this account, the outgoing transfer reconciles its share of her total BUCK-age against the Jubilee: roughly $0.02 \cdot 500 = 10$ BUCK worth of demurrage is paid out of her balance (the exact amount depends on her other holdings' age). Net effect: she receives 1,000, pays ~ 10 when she spends them, nets 990. This matches the previous-model result where she received 990 at redemption – same economic total, different accounting.

Now suppose an anonymous donor sends 500,000 BUCK to the contract address. B rises to 10,500,000; the donor's BUCK-age stays with them. The pool's A/B drops from 0.5 to $5,000,000 / 10,500,000 \approx 0.476$. Alice's next redemption absorbs a smaller age-share per face – a direct subsidy from the donor.

No Jubilee payment was emitted by the pool during any of this. The Jubilee's growth is unchanged: it keeps accruing $\kappa \cdot \text{totalSupply}$ per block regardless of who holds which BUCK.

6.7 Who Bears the Demurrage, Really

The answer is now sharply localized: **the Note recipient bears the pool's accumulated share of demurrage on the face value they received, and they pay it when they next spend from that account.** The recipient has accepted the note, and with it has accepted the demurrage cost of the BUCK sitting in the pool while the note was outstanding.

This is the "price" the recipient pays for accepting BUCK Notes instead of a direct BUCK transfer: they inherit the note's pool-residence demurrage, prorated across all live notes at the

moment of redemption. The trade-off is explicit – and visible in the resulting (B, A) state on the recipient’s account.

Noteholders (while holding) bear no demurrage: they hold a cryptographic note, not a BUCK balance, and the BUCK they implicitly own sits in the pool where its age is borne collectively. The issuer (at mint time) bears their own outgoing-transfer demurrage to deposit into the pool, as for any ERC-20 transfer. Third parties bear nothing. The Jubilee keeps receiving the BUCK-wide demurrage stream $\kappa \cdot \text{totalSupply}$ regardless of how BUCK sloshes between accounts.

DRAFT

7 Demurrage Solves the Lost-Cash Problem

This section is about a property that is genuinely novel, with implications well beyond the BUCK system.

7.1 The Hidden Inflation of Lost Cash

Every monetary system that uses physical bearer instruments suffers a constant slow loss to *unrecoverable currency*. Banknotes are lost in fires, dropped down storm drains, forgotten in the pockets of jeans that get washed and shredded, buried in landfills, sealed in walls, hidden by the elderly and forgotten by their heirs.

The Federal Reserve estimates that 5-10% of US paper currency is permanently unrecoverable at any given moment.¹¹ At a US M0 of approximately \$2.3 trillion, this represents *\$115-230 billion in permanently extracted value* – seigniorage to the Federal Reserve, since the lost notes never get redeemed but the backing assets remain on the Fed’s books.

This is not theft, but it is not neutral either. Lost cash is a regressive transfer: the people most likely to lose cash are the people who use cash most (the elderly, the unbanked, low-income workers paid in cash). The beneficiary is whoever issued the cash, generally the central bank, ultimately the government.

7.2 What BUCK Demurrage Does For Lost Notes

A BUCK Note that is lost in a fire, dropped down a storm drain, or hidden in a wall by a paranoid grandparent is *not* permanently extracted from the system. The face value v sits in the pool as part of B , earning BUCK-age A at the standard rate. No redemption ever happens.

Two effects balance:

1. The BUCK backing the lost note stays in the pool indefinitely, accruing age. It does not vanish.
2. Other live-note redeemers, through the age-preserving transfer, absorb an age-share $v \cdot A/B$ at each deposit. Because the lost note’s share weighs on A without a corresponding redemption to drain it, the lost note’s implicit demurrage is paid installment-style by subsequent redeemers.

Over decades, the pool’s age-per-BUCK A/B drifts upward as lost notes accumulate unredeemed. The autonomous Jubilee growth at rate $\kappa \cdot \text{totalSupply}$ continues regardless. Net effect: *the 5-10% of currency that physical cash systems lose forever, BUCK Notes returns to the commons within a generation*, via the steady demurrage pressure that the lost face value applies to live redeemers.

This has cascading consequences:

1. **Money supply self-correction.** No system-wide accounting tail of "missing" currency that the central authority must track and write off. Lost notes write themselves off through the pool-age mechanism.

¹¹Federal Reserve Bank of San Francisco. "Diary of Consumer Payment Choice" (2018). Estimates 5-10% of US paper currency by value is permanently unrecoverable. At a US M0 of \$2.3 trillion, this represents \$115-230 billion in permanently extracted seigniorage.

2. **Anti-hoarding incentive matches on-chain balances.** A BUCK held in your wallet or in your pocket as paper decays at the same rate (via the age-share that will eventually be absorbed on redemption). There is no advantage to converting on-chain BUCK to paper to evade demurrage; the demurrage clock runs the same way in both forms.
3. **Inheritance is automatic.** A grandparent who hides 100,000 BUCK in paper notes and forgets to tell anyone does not extract 100,000 BUCK from the economy on their death. The value flows back through the Jubilee to BUCK issuers – broadly, back into the same economy where the grandparent originally earned it.
4. **No central bank profit from currency loss.** The seigniorage that accrues to central banks today from unrecoverable cash does not exist in BUCK Notes. Lost value does not benefit the issuer; it benefits the system as a whole.
5. **Counterfeit recovery is automatic.** Even if a note printed years ago were somehow forged (which is cryptographically impossible, but humour the counterfactual), the value at stake is bounded by the demurrage decay. An attacker forging old notes would be forging paper whose backing BUCK has already been largely absorbed by live redemptions.

This is, I believe, the first monetary system in history where lost cash returns to the commons by automatic mechanism rather than by accounting fiat. Physical cash cannot have this property, because physical cash has no clock. It emerges naturally from BUCK's existing demurrage mechanism applied to the note-pool primitive.

7.3 The Symmetry With On-Chain Balances

A second elegance worth pointing out: the effective demurrage rate on a BUCK Note – as absorbed by the recipient at deposit – is *the same* as the demurrage rate on an on-chain BUCK balance that lived in the pool for the same duration. This is by design, and it matters.

If paper BUCK Notes demurraged faster than on-chain BUCK, holders would prefer on-chain (and paper would be discouraged, defeating the design's purpose). If on-chain BUCK demurraged faster than paper notes, holders would race to print all their on-chain balances as physical notes (and the chain would lose its primary use as a settlement layer).

The matching demurrage rate makes the choice between on-chain and paper purely about *use case fit*: on-chain for routine commerce, automated payments, large transactions; paper for offline payments, gifts, savings under the mattress, disaster preparedness, and so on. Neither form is privileged by the demurrage mechanism.

8 Long-Term Holding Safety: The Printer-Trust Distinction

Pool-rate demurrage makes every note type equally cheap to *hold*. What distinguishes them for long-term storage is not demurrage but the *printer-trust* model: who, besides the intended holder, retains information that could let them spend the note?

8.1 The Printer-Trust Model

At mint time, the issuer knows:

- The note's commitment preimage $(v, \rho, \text{id}_{\text{payload}}, \text{predicate})$
- Whatever additional secret is packed into $\text{id}_{\text{payload}}$ (the issuer's sk_{issuer} for Type 1, the ephemeral r_e for Type 3, etc.)

The question is whether this knowledge enables the issuer – or anyone else with the same knowledge – to spend the note later.

8.2 Type 1 (Encrypted Issuer Identity): Safe

The spend predicate requires a Chaum-Pedersen proof of equality with the depositor's *registered* credential. The issuer is the intended eventual depositor (possibly via a different registered account sharing the same M). No third party can spend: even with full knowledge of ρ , a non-issuer's registered credential does not encrypt M_{issuer} , so the CP relation fails.

Long-term holding is safe. The only person who can spend is the issuer, and the issuer is the intended holder.

8.3 Type 2 (Public Issuer Identity): Unsafe for Long-Term Storage

The spend predicate requires only knowledge of ρ plus any registered identity. The issuer minted the note, so the issuer knows ρ . If the issuer retains any copy of ρ after printing the physical note – in a database row, a printed backup, a backup transaction – the issuer can race the holder to deposit.

This is not a cryptographic failure; it is a policy failure. A trustworthy issuer *could* destroy ρ after printing. But the *holder has no way to verify that destruction*. The note is only as safe as the issuer's operational discipline.

Long-term holding is unsafe. Type 2 notes should be cashed promptly (hours to days). They are a cheque, not a savings instrument. In the failure mode where the issuer does retain ρ , legal recourse is via the issuer's registered Identity – which is the point of making the issuer *public*: they cannot deny having printed the note, and the registered Identity is subpoena-able.

For use cases that need long-term safety with public attribution (e.g., an inheritance instrument where the issuer's identity must be known), the natural solution is to use Type 3 (stealth-targeted) and separately publish the issuer's identity in a signed accompanying statement rather than bake it into the commitment.

8.4 Type 3 (Stealth-Targeted to Recipient): Safe

The spend predicate is a Schnorr PoK of the stealth secret $sk^* = H_{\text{st}}(s) \cdot sk_{\text{recipient}}$. Computing sk^* requires $sk_{\text{recipient}}$, which only the recipient possesses. The issuer knows r_e and can compute $s = r_e \cdot pk_{\text{recipient}}$ and $H_{\text{st}}(s)$, but *cannot* compute $sk^* = H_{\text{st}}(s) \cdot sk_{\text{recipient}}$ without $sk_{\text{recipient}}$ (Theorem 7, Corollary 1 – "printer non-fungibility").

Long-term holding is safe. The issuer’s knowledge of every mint-time secret does not enable them to spend. Only the intended recipient, through their own sk , can redeem.

8.5 Summary: Which Notes Can Be Saved?

Note type	Issuer can spend later?	Long-term holding safe?	Comparable instrument
Type 1	Only to themselves	Yes	Savings account (self-owned)
Type 2	Yes (knows ρ)	No	Uncashed cheque (needs to clear)
Type 3	No (Theorem 7)	Yes	Registered bond (recipient-locked)

The consequence: *only Type 1 and Type 3 notes are suitable for long-term holding.* A user printing a Type 2 cheque and placing it in a safe deposit box for a decade is, cryptographically, trusting the issuer’s operational discipline for a decade; a user doing the same with Type 1 or Type 3 is trusting only the cryptography.

This constrains how the note types map to use cases:

- **Long-term savings, cold storage, estate, self-wormhole:** Type 1 (if you are your own beneficiary) or Type 3 (if a named heir is the beneficiary).
- **Cheques, payroll, invoices:** Type 2, cashed promptly.
- **Targeted gifts, remittances, inheritance:** Type 3.
- **Self-wormhole cross-address transfers:** Type 1 exclusively.

9 Physical Security Model

The cryptography only protects what the cryptography touches. The physical paper itself needs a security model. This section sketches one.

9.1 Layered Defense

A printed BUCK Note has three concentric layers of security:

Layer	What it protects	Mechanism
Inner: secret	The cryptographic ability to spend	QR code containing ρ
Middle: physical seal	Type 2 publication-exclusivity	Hologram, scratch-off, laminate sleeve
Outer: identity binding	Type 1/Type 3 targeting (only intended spender)	Stealth address $pk^* = H_{st}(s) \cdot pk_{\text{recipient}}$ or CP-equality

The *inner* layer is the cryptographic core: as long as ρ (and the type-specific secret material packed into $\text{id}_{\text{payload}}$) is unknown, the note cannot be spent. This is unconditional protection against any threat that does not include observing those secrets.

The *middle* layer matters only for Type 2 notes, where the spender predicate is "anyone who knows ρ plus any registered identity." An unsealed Type 2 QR photographed by any observer allows that observer to race to spend. Sealing the QR under a tamper-evident covering means the note can be displayed (passed hand-to-hand, stored in a wallet, mailed in an envelope) without exposure. The seal must visibly damage on opening so that recipients can verify they were the first to expose the secret. (This does not rescue Type 2 from the issuer-race risk – the issuer who printed the note could still have retained ρ before sealing – which is why Type 2 remains unsafe for long-term holding. The seal addresses post-printing observation, not issuer trust.)

The *outer* layer makes Type 1 and Type 3 notes safe to *publicly* distribute. A Type 1 or Type 3 note can be photographed with no security implication: the photographer cannot spend it without also possessing the stealth secret or the matching Identity scalar. The targeting binding is cryptographic, not physical. For Types 1 and 3, no physical seal is required.

9.2 Recommended Manufacturing Profile

A practical BUCK Note specification, suitable for industrial-scale printing:

- **Substrate.** Cotton-blend banknote paper or polymer (Bank of Canada-style). Steel or titanium plates for high-denomination "permanent" notes.
- **Printing.** Standard inkjet or laser printing of QR codes is sufficient for cryptographic correctness. For visual anti-confusion (so people can tell a legitimate note from a printout), use UV-reactive ink, microprinting, or watermarks – the same techniques used for traditional banknotes. These do not add cryptographic security; they add *reassurance*.
- **Seal.** Peelable hologram covering the secret QR (Type 2 only), or scratch-off panel covering a separate code (also Type 2), or no seal at all (Type 1 and Type 3, since the cryptography handles exclusivity).
- **Redundancy.** Print the QR twice (front and back) with Reed-Solomon error correction. A note that loses one QR to wear or damage can still be deposited via the second. For higher-value notes, also print the secret as base32 digits in human-readable form.

- **Issuer mark.** A non-cryptographic logo identifying the issuer (the Alberta Buck reserve, a credit union, a commercial issuer). This is for visual recognition only; the cryptographic authenticity check is the on-chain commitment.
- **Denomination indication.** Visible face value for human convenience. The actual value is what the on-chain commitment encodes; the printed face value is metadata. A wallet app that scans an apparent 1,000 BUCK note and reads "actual value: 100 BUCK" from the commitment warns the user before deposit.

DRAFT

10 Use Cases

The same primitive serves a spectrum of use cases that today require entirely separate financial instruments. Each use case maps to one or two of the three Identity-binding types, with printer-trust implications determining which choice is appropriate.

10.1 Walking-Around Money

Print a stack of Type 1 notes issued from one of your own private accounts, in small denominations (10, 20, 100 BUCK). Hand them out as payment; they can only be deposited by *you*, at one of your other registered private accounts.

This differs from physical cash: a Type 1 note is *not* pure bearer – a stranger who picks up a dropped Type 1 note cannot spend it. For a true cash-equivalent (spendable by anyone who holds the paper), a Type 2 note with hologram seal fills the role, at the cost of short-term expiry and issuer-trust. Most walking-around use cases – paying for a coffee, tipping, splitting a bill with a friend – are better served by Type 1 notes that you print for your own use and exchange via direct digital transfer of the opening at the moment of payment.

Notes age at the pool rate, encouraging rapid circulation rather than hoarding. The wallet-app status codes give immediate VALID / DEAD / FAKE feedback on scan.

10.2 Cross-Border Remittances

A paper BUCK Note is just paper. It crosses borders in pockets, envelopes, and parcels at zero marginal cost. A Type 3 note targeted to the recipient's Identity makes the transfer useless to anyone except the intended recipient, so loss in transit is not a financial risk.

This compares favorably to the existing remittance industry, which extracts roughly 6.5% in fees globally on \$700+ billion in annual transfers.¹² A paper-note remittance has a fixed cost: print

- mail. The mail might cost \$1; print costs fractions of a cent;

deposit gas costs a few dollars at most. At any reasonable remittance value, the fee compression is enormous.

The recipient does not need a smartphone or internet at the moment of receipt – they need them only at the moment they want to convert the note to on-chain BUCK or to local fiat. In countries with intermittent connectivity, this is a major usability improvement over digital remittance services.

10.3 Tip Jars and Charitable Donations

A sheet of small-denomination Type 2 notes, perforated for tear-off, can be left next to a tip jar or a donation box. Patrons take notes (denominated at, say, 2 BUCK each), scan them with their phone, and the value flows to the jar's registered identity.

This works without any intermediary, payment processor fee, or opening of an account. The patron pays in paper; the recipient receives in on-chain BUCK. Type 2 is appropriate here because the turnaround is fast: the jar's operator empties the day's receipts to their main account nightly.

¹²World Bank. "Remittance Prices Worldwide" (Q4 2024). Average global cost of remitting \$200 was approximately 6.5% in 2024. Total global remittance flows estimated at \$702 billion in 2024.

10.4 Cold Storage and Disaster Preparedness

A Type 1 self-issued note, stored in a fireproof safe, is a more durable form of cold storage than a 12-word seed phrase. The note is *self-evidently* an instrument of value – a customs officer, a relative, or your future self immediately recognizes a paper labeled "1,000 BUCK Note" as something worth keeping safe. A seed phrase, by contrast, is opaque to anyone who has not been trained to recognize it.

For disaster preparedness: a small folder of BUCK Notes in a fireproof safe survives infrastructure failures that would render exchange accounts and even hardware wallets unusable. As long as the chain is reachable somewhere in the world, the notes can be deposited. In a true grid-down emergency, the notes circulate hand-to-hand at face value (with the matching pool-rate demurrage clock running uniformly, so prices remain stable in BUCK terms even during a multi-week outage).

Type 1 is the correct choice for self-directed cold storage (you are your own beneficiary). Type 3 targeted to a designated heir's Identity is the correct choice for estate-directed cold storage.

10.5 Estate Planning

Type 3 notes targeted to your heir's Identity, stored in a safe-deposit box: zero probate process, zero tax-event delay, zero court intervention. The heir scans on your death, deposits, the BUCK is theirs.

For larger estates, additional conditional-spend predicates (time locks, death-certificate oracle attestation) can be composed into the commitment's `predicate` field. The SNARK does not care what the predicate is as long as it is well-defined at mint time.

10.6 Gift Cards Without Issuer Risk

A traditional gift card carries the risk that the issuing merchant goes bankrupt between purchase and redemption (cumulative consumer losses to bankrupt gift-card issuers run into the hundreds of millions annually in North America¹³). A Type 1 or Type 3 BUCK Note carries no issuer risk: the value is on chain in the BUCK reserve, not on the merchant's books.

Specialized merchant gift cards become unnecessary – replaced by BUCK Notes of appropriate denomination, optionally bundled with a paper merchant receipt indicating the intended use.

10.7 Tax-Compliant Compensation

An employer paying a contractor in BUCK Notes faces the same tax-reporting obligations as paying in cash: payment is income and must be reported. The on-chain issuance trail makes this easier to comply with than paper cash, not harder – the employer's mint transaction is an auditable record of the payment, even though the contractor's deposit is private.

For employees: employers can issue Type 3 notes targeted to each employee's registered Identity, automating payroll while preserving employee privacy from external observers.

10.8 Underbanked and Cash-Economy Inclusion

This is, perhaps, the most important use case.

¹³Consumer Federation of America. "Gift Cards: A Costly Convenience" (2023). Estimates of cumulative consumer losses to bankrupt gift-card issuers in the United States range from \$200M-\$1B annually depending on methodology.

The Bank of Canada estimates 1-2% of Canadians are unbanked, with rates an order of magnitude higher in rural and Indigenous communities.¹⁴ Globally the World Bank estimates 1.4 billion adults are unbanked.¹⁵

For these populations, paper cash is not a privacy preference; it is the only viable financial instrument. A digital-only currency (CBDC, stablecoin, traditional e-payment) excludes them by definition. BUCK Notes includes them: a person without a bank account, without a smartphone, without literacy in the local language, can hold and transfer BUCK Notes by physical handover, exactly as they hold and transfer paper currency today.

When and if they choose to interact with the digital economy, they deposit a note and the value moves on chain. Until then, they participate in the BUCK economy on exactly the terms they participate in the cash economy. The issuer of the paper – a credit union, a reserve-backed issuer like ATB, or a local community organisation – takes on the role that the central bank currently plays for physical fiat, while the recipient retains the anonymity of a cash user.

DRAFT

¹⁴Bank of Canada. "2022 Methods-of-Payment Survey" (2023). Approximately 1-2% of Canadians lack bank accounts; rates substantially higher in rural and Indigenous communities.

¹⁵World Bank. "The Global Findex Database 2021" (2022). An estimated 1.4 billion adults globally remain unbanked, with disproportionate concentrations in low-income countries, women, rural populations, and the elderly.

11 Selective Disclosure: Compliance Without Surveillance

Bearer instruments are politically controversial. Most jurisdictions either regulate them heavily (US bearer bonds were effectively abolished in 1982 by the Tax Equity and Fiscal Responsibility Act) or enforce strict reporting thresholds on large cash transactions (Canada's FINTRAC reports any cash transaction above \$10,000).

The concern is not bearer instruments per se; it is the *opacity to law enforcement and tax authorities* of large bearer-instrument flows. Money laundering, sanctions evasion, and tax evasion all exploit bearer-instrument anonymity.

BUCK Notes threads this needle in a way that paper cash and CBDCs both fail.

11.1 The Asymmetry: Issuance vs. Circulation

Every BUCK Note's *issuance* is on chain and identity-bound. When Alice mints 5,000 BUCK from her registered Identity, the chain records:

- Alice's Identity (via the existing PS signature + ElGamal credential machinery)
- The amount transferred into the Note Pool
- The block timestamp
- The commitments appended to the tree

This record is *permanent* and *cryptographically traceable to Alice's Identity*.

The *circulation* of those notes – who holds them, who passes them on, who eventually deposits them – is private. The chain sees only the deposit transaction, which reveals the recipient's address (the depositor) and the amount, but nothing about the path from issuer to depositor.

This produces an asymmetry:

- *Anyone* can see that "5,000 BUCK was minted as notes by [some identity] at block 1,234,567"
- *Only Alice or a regulator with Alice's selective-disclosure key* can see that "the issuer was Alice"
- *Nobody* can see who held the notes between issuance and deposit
- *Anyone* can see "[some address] deposited a 1,000 BUCK note at block 1,234,890"

Linking issuance to deposit requires either temporal correlation (defeated by a sufficiently large global note pool) or knowledge of the opening (which only a holder of the note has at any given time).

11.2 The Regulatory Audit Path

A regulator investigating illicit flows has a clean audit path:

1. Identify a suspect deposit on chain (via subpoena to the depositing party's Identity, KYC obligations on the depositing wallet operator, etc.)
2. Subpoena the issuer of any BUCK held by the suspect: the *issuer* is identifiable from the on-chain issuance record (selective disclosure of the issuer's Identity, through the existing `approveFor` pattern on the issuance transaction)

3. Examine the issuer’s records for who they paid the note to (this is the issuer’s responsibility, the same way a bank that issues a cashier’s cheque is responsible for knowing who they issued it to)

The audit path stops at the first hand-to-hand transfer (which is intentionally private, as physical cash is private). But the *origination* of every note is traceable in a way that physical cash is not – a \$20 bill issued by the Bank of Canada is identical to every other \$20 bill, with no per-bill issuance trail at all.

11.3 What Compliance Looks Like

A reasonable regulatory regime around BUCK Notes:

- *Issuers* (anyone who mints notes from on-chain BUCK) must be identity-registered and may face reporting obligations on large issuances (analogous to current cash-withdrawal reporting).
- *Depositors* (anyone who deposits a note to mint on-chain BUCK) face the same reporting obligations as cash deposits at a bank today.
- *Holders* who simply pass notes hand-to-hand have no reporting obligation, the same way you have no obligation to report having \$200 in your wallet.
- *Pool-rate demurrage* makes large-scale long-term cash hoarding economically unviable (the demurrage decay caps the value of cash-as-store-of-value at roughly the same level as a moderate-yield savings account loses to inflation).

This regime is *strictly more privacy-preserving* than the current Canadian-dollar regime (which surveils every electronic transaction) and *strictly more compliance-friendly* than the current paper-cash regime (which provides no issuance trail at all).

11.4 The CBDC Comparison

China’s e-CNY, the European Central Bank’s proposed digital euro, the Bank of Canada’s CBDC research – all of these designs include either explicit or implicit surveillance capabilities. The central bank can see, freeze, expire, restrict, geofence, or retroactively examine any holder’s balance.²

This is not an accident of the technology; it is a design goal. The pitch to governments is precisely that CBDCs give them visibility into civilian payment flows that paper cash denies them.

BUCK Notes is the opposite design. Visibility is at the edges (issuance, deposit), not the center (circulation). The issuer is identifiable, but only by selective disclosure with proper authorization (subpoena, court order, regulatory audit authority), not by everyday observation. The bearer-circulation phase is private the way cash is private.

A jurisdiction that adopts BUCK Notes gets *better compliance tools than paper cash provides today* without giving up *the privacy properties that make cash worth using*. This is a genuine Pareto improvement on the current regulatory trade-off and would, I believe, be politically defensible to even cash-skeptical authorities.

12 Comparison With Prior Art

This section places BUCK Notes in the lineage of digital cash and privacy-preserving pool designs from 1982 to the present, and identifies what each predecessor lacked.

12.1 Chaum's E-Cash (1982-1998)

David Chaum's blind-signature scheme⁴ introduced the core idea: an issuer (a bank) signs a token blinded by the holder so that the bank cannot link the issuance to the redemption. When the holder spends the token, the merchant verifies the signature without revealing who originally received it.

This is a beautiful design and operationally similar to BUCK Notes in spirit. Its failures were *institutional*, not cryptographic:

- Required a centralized issuer (the bank)
- Required online verification at spend time (no offline transferability)
- Two-sided market never bootstrapped (no merchants accepted, so no consumers held)
- DigiCash (the commercial implementation) went bankrupt in 1998

BUCK Notes has the same privacy properties without the centralized issuer (any identity-registered party can mint notes against their own on-chain BUCK). Spend-time verification is on chain rather than via the issuer (so the issuer cannot selectively block redemptions). The two-sided market problem is mitigated by BUCK's existing on-chain ecosystem – BUCK already exists as a usable currency, so adopting BUCK Notes is a UX upgrade rather than a greenfield bootstrapping.

12.2 Brands' Offline Divisible E-Cash (1993)

Stefan Brands' scheme⁵ addressed the offline-spend problem: a divisible coin can be spent in pieces without contacting the issuer, but *double-spending mathematically reveals the spender's identity*.

This is the "deterrent rather than prevention" model. It works in theory but has two operational issues:

- The deterrent only deters if identity revelation has consequences (it relied on a legal/social enforcement layer external to the cryptography).
- The complexity of divisible-coin management was substantial and never produced a shipped product.

BUCK Notes handles double-spend by *prevention* (global nullifier set), not deterrence. No legal layer is required. Divisibility is achieved by issuing multiple notes at mint time rather than by a single divisible coin.

12.3 Casascius Coins (2011-2013)

Mike Caldwell's Casascius coins³ were physical Bitcoin: a brass or silver coin with a holographic seal covering a private-key QR code. Snap the seal, scan the QR, the coin's Bitcoin balance was yours.

Casascius coins remain the closest existing analogue to paper BUCK Notes. What they lacked:

- Native integration with the underlying token (the Bitcoin protocol does not understand bearer-coin form; the coin is an aftermarket wrapper).
- Targeting (all coins were pure bearer; no way to issue a coin claimable only by a specific recipient).
- Demurrage (a Casascius coin holds its full Bitcoin balance forever, unrelated to any economic mechanism).
- Programmable predicates (only one mode: bearer).
- Issuer was a single individual, who eventually stopped issuing in 2013 due to FinCEN money-transmitter concerns – an issuance bottleneck that BUCK Notes distributes to all identity-registered participants.

BUCK Notes is, in essence, what Casascius would have been if Bitcoin had been designed with bearer-paper as a first-class instrument from the start, and if the underlying token had a demurrage and identity layer.

12.4 BIP-38 Paper Wallets

Encrypted Bitcoin private keys printed on paper, with a passphrase needed to spend. BIP-38 paper wallets are an aftermarket bearer instrument similar to Casascius but without the physical coin form factor.

Same limitations as Casascius (no native integration, no targeting, no demurrage, no predicates), plus the additional UX burden of remembering or transmitting the passphrase.

12.5 Zerocash / Zcash (2014-)

Ben-Sasson, Chiesa, Garman, Green, Miers, Tromer, and Virza’s Zerocash⁷ gave us the first scalable on-chain commitment-and-nullifier UTXO model with full zero-knowledge. Zcash deployed it in 2016.

Zcash provides on-chain shielded transactions but does *not* extend the model to physical bearer instruments, targeted cheques, or identity-linked issuance. All Zcash value lives on chain in shielded notes. No mechanism exists for printing a shielded Zcash note as a paper instrument that someone else could deposit later.

BUCK Notes uses Zerocash’s commitment-and-nullifier primitive for its on-chain machinery and extends it with the Identity-binding layer and the physical-paper surface. The contribution is in the extension, not the primitive.

12.6 Aztec / Zk.Money

Aztec’s predicate-encrypted notes⁸ generalize Zerocash to private smart contract calls – a note can encode arbitrary state operated on by arbitrary predicates. This is the cryptographic ancestor of the **predicate** field in the BUCK Note commitment.

Aztec does not have physical bearer instruments either. Its predicates are designed for on-chain DeFi applications (private DEX trades, private lending), not for paper notes or identity-bound cheques. BUCK Notes applies the same predicate machinery to a different problem: specifying spend conditions for a private bearer instrument that is issued against an identity-registered reserve.

12.7 Tornado Cash and Privacy Pools

Tornado Cash⁹ was the simplest possible commitment-and-nullifier mixer: fixed-denomination deposits in, anonymous withdrawals out. Sanctioned by OFAC in 2022 for money-laundering use.

Privacy Pools (Buterin, Schaefer, Tromer, et al., 2023)¹⁰ extends Tornado with per-spend "association sets" – a depositor can prove their deposit is in a non-incriminating subset, allowing compliance without identification.

Both are mixer designs, not cash-instrument designs. Their relevance to BUCK Notes is the lineage: the Note Pool is a Tornado-style global anonymity set, but with predicates and Identity bound in. The selective-disclosure compliance story borrows from Privacy Pools' association-set ideas, but with issuer identity substituting for the depositor-declared non-incrimination proof.

12.8 Comparison Summary

System	Bearer paper	Targeting	Demurrage	Predicates	Native	Decentralized	Year
Chaum E-Cash	No	No	No	No	-	No	1982
Brands Offline	No	No	No	No	-	No	1993
BIP-38 Paper Wallet	Yes	No	No	No	No	Partial	2010
Casascius Coin	Yes	No	No	No	No	No	2011
Zerocash / Zcash	No	No	No	No	-	Yes	2014
Tornado Cash	No	No	No	No	-	Yes	2019
Aztec	No	No	No	Yes	-	Yes	2020
Privacy Pools	No	No	No	Limited	-	Yes	2023
CBDCs (e-CNY, etc.)	No	Yes	Sometimes	No	Yes	No	2020+
BUCK Notes	Yes	Yes	Yes	Yes	Yes	Yes	2026?

The "Native" column captures whether the bearer-paper instrument is a first-class designed primitive of the system (as opposed to an aftermarket wrapper or absent entirely). BUCK Notes is the first design that fills every column. This is not because the prior art was incompetent; it is because the four required pieces (commitment-pool primitive, identity layer, demurrage, jurisdictional reserve) finally converged in one system.

12.9 Comparison: Cash, CBDC, BUCK Notes

A more direct comparison aimed at a non-cryptographer reader: how does BUCK Notes stack up against physical cash and central bank digital currencies?

Property	Physical Cash	CBDC (typical)	BUCK Notes
PRIVACY			
Holder's identity hidden	Yes	No (issuer sees all)	Yes (during circulation)
Transaction graph hidden	Yes	No	Yes
Issuer's identity hidden	Yes (notes are fungible)	No	No (selective disclosure)
FINALITY			
Settlement is instant	Yes	Yes	Yes (chain finality)
Reversible by issuer	No	Yes (CBDC freeze)	No
Reversible by holder	No	Yes (chargeback)	No
INFRASTRUCTURE			
Works without internet	Yes	No	Yes (transfer); needs net for deposit
Works without battery	Yes	No	Yes (paper); needs power for deposit
Works without bank account	Yes	No	Yes
Cross-border in your pocket	Yes (with declaration)	No (geofenced)	Yes
DURABILITY			
Survives infrastructure failure	Yes	No	Yes (paper); chain on recovery
Counterfeit-resistant	Partial (printed security)	Yes	Yes (cryptographic)
Recoverable if lost	No	Yes (issuer can re-issue)	No (but demurrage returns value to system)
ECONOMIC BEHAVIOR			
Lost notes extracted forever	Yes	No	No (returned via demurrage)
Inflation tracks issuer policy	Yes	Yes	Yes (tracks BuckK)
Holds value over decades	No (inflation)	No (inflation)	No (demurrage)
Holds value over years	Approximately	Approximately	Yes (with demurrage discount)
PROGRAMMABILITY			
Time-locked spends	No	Possible	Yes
Conditional payouts	No	Possible	Yes
Multi-signature spending	No	Possible	Yes
Issuer can restrict spending	No	Yes	No (issuer relinquishes control at mint)
COMPLIANCE			
Issuance is auditable	No	Yes	Yes
Circulation is auditable	No	Yes	No (private)
Deposit is auditable	Partial (CTR rules)	Yes	Yes
Selective disclosure available	No (no records)	No (full disclosure)	Yes

The dominant insight from this table: *BUCK Notes combines the privacy properties of physical cash with the compliance and durability properties of CBDC, while preserving infrastructure independence that CBDC abandons.*

The single property where physical cash is genuinely irreplaceable – "works without internet for deposits" – is moderated for BUCK Notes but not eliminated. Holders can transfer BUCK Notes hand-to-hand offline, indefinitely; they need internet only when they want to convert the bearer instrument to an on-chain balance. In a disaster scenario, BUCK Notes circulate as cash for the duration of the outage, then deposit normally when the chain becomes reachable again.

13 Required Identity Cryptography

BUCK Notes re-use the existing Identity machinery (Identity) wherever possible, adding one new note-side primitive and one re-use of the Theorem 7 stealth-address construction.

13.1 Components Already Present

The following are unchanged from Identity v1 and are used directly:

1. **Pointcheval-Sanders issuer signature** σ on the Identity scalar m . Re-randomisation continues to be used for unlinkable credential generation (Identity Fountain). No change.
2. **ElGamal registered credential** $E = (R, C) = (rG, M + r \cdot pk)$ where $M = m \cdot G$. Continues to be the on-chain identity witness. No change.
3. **Chaum-Pedersen equality of discrete logs**. Used by Type 1 spend predicate for the same- M proof; identical to the **approve** flow's CP proof, just wrapped in a SNARK. Theorem 6 applies unchanged.
4. **Schnorr PoK of sk for registered pk** . Used by Type 2 and Type 3 spend predicates to bind the depositing account to a registered identity. Single-statement case of CP, Theorems 3-5 apply verbatim.
5. **IdentityRegistry** mapping between on-chain addresses and registered credentials (PS signatures, ElGamal ciphertexts, public-claim flags).

13.2 New Primitive: The Note Commitment Hash

A SNARK-friendly commitment hash H_{cm} (Poseidon) is needed to compute the commitment inside the spend circuit. The hash is deterministic, collision-resistant over the note's field, and efficient in R1CS; Poseidon over the BN254 scalar field is the natural choice and is already used by the Aztec and Zcash Sapling circuits.

No novel cryptographic construction; only a circuit-level primitive that is standard in modern SNARK systems.

13.3 New Primitive: The Nullifier PRF

The nullifier PRF $\text{PRF}_{\text{key}}(\rho)$ is also Poseidon-based, keyed differently per type:

- **Type 1:** key = sk_{issuer} (which, per Theorem 6, equals the depositor's sk since same M). Nullifier is therefore computable by the depositor and uniquely determined by ρ and the Identity scalar shared between issuer and depositor.
- **Type 2:** key = ρ itself; nullifier = $H_{\text{nf}}(\rho)$, publicly computable by anyone who knows ρ . This is the reason Type 2 is vulnerable to issuer-race: whoever knows ρ can compute the nullifier and race a deposit.
- **Type 3:** key = sk^* (the stealth secret, known only to the recipient). The nullifier is uniquely bound to (sk^*, ρ) ; only the intended recipient can compute it.

Cross-type nullifier-space separation is enforced by a type-domain-separator byte in the PRF input, so collisions across types are avoided.

13.4 Re-Used Primitive: Theorem 7 (Stealth Address)

The stealth-address construction $pk^* = H_{st}(s) \cdot pk_{\text{recipient}}$ is proved sound in the proofs document (Theorem 7) under DLog on G_1 alone. The same construction is used here verbatim for Type 3 notes. No modification, no new assumption.

13.5 The Re-Encryption Operation at Mint

Type 1 and Type 3 require the issuer to perform a re-encryption operation at mint time:

Type 1 mint-time re-encryption. The issuer takes their own registered $M_{\text{issuer}} = m_{\text{issuer}} \cdot G$ and produces $E_{\text{issuer}}^{(\text{note})} = (r_i G, M_{\text{issuer}} + r_i \cdot pk_{\text{issuer}})$ with fresh r_i . This is just a fresh ElGamal encryption under the issuer's existing pk; the same operation Alice performs at **approve** time.

Type 3 mint-time stealth derivation. The issuer picks $r_e \leftarrow \mathbb{Z}_q^*$, computes $R_e = r_e G$, computes the shared point $s = r_e \cdot pk_{\text{recipient}}$ (the issuer can do this because she knows r_e and $pk_{\text{recipient}}$), and computes $pk^* = H_{st}(s) \cdot pk_{\text{recipient}}$. She packs (R_e, pk^*) into the commitment. She does *not* compute $sk^* = H_{st}(s) \cdot sk_{\text{recipient}}$: she can't, because she lacks $sk_{\text{recipient}}$.

Both operations are single-curve-multiplication and single-hash operations on BN254 G_1 . No pairing, no new hardness assumption.

13.6 The Re-Encryption Operation at Spend (Type 1 Only)

A Type 1 depositor must produce a fresh ElGamal encryption $E_{\text{dep}} = (r_d G, C_d) = (r_d G, M_{\text{issuer}} + r_d \cdot pk_{\text{dep}})$ and a SNARK-internal CP proof that $E_{\text{issuer}}^{(\text{note})}$ and E_{dep} decrypt to the same M . This is Theorem 6 (end-to-end re-encryption correctness) lifted into the SNARK circuit: the statement proved is identical to the on-chain CP proof from **approve**, just encoded as a SNARK witness rather than a Fiat-Shamir NIZK.

The extractable witness is $(sk_{\text{issuer}}, r_d)$, and since sk_{issuer} is bound (by registration) to M_{issuer} , the proof establishes that the depositor owns the registered credential that encrypts the same M the issuer encrypted.

13.7 The Schnorr PoK Operation at Spend (Type 2 and Type 3)

Both Type 2 and Type 3 spenders include a Schnorr PoK of sk_{dep} for their registered pk_{dep} , Fiat-Shamir bound to the spend transaction. For Type 3, an *additional* Schnorr PoK of sk^* for pk^* is required, as per Theorem 7.

Both Schnorr proofs are standard; no novelty.

13.8 Summary: New Cryptographic Components

Component	Status	Assumption
Poseidon commitment hash	New circuit primitive (standard in SNARKs)	ROM-like
Poseidon nullifier PRF	New circuit primitive	ROM-like
PS signature (issuer)	Re-used from Identity	q-SDH
ElGamal credential + re-encryption	Re-used from Identity	DLog
Chaum-Pedersen equality proof	Re-used (lifted into SNARK) from Identity	DLog + ROM
Stealth address derivation	Re-used from Proofs Theorem 7	DLog + ROM
Schnorr PoK of sk	Re-used from Identity	DLog + ROM
SNARK proof system (Groth16/PLONK)	Standard on BN254	system-dep

Only the Poseidon hash family is new, and it is already standard in the SNARK ecosystem. No novel hardness assumption is introduced.

14 The SNARK Circuits

Three spend circuits (one per note type) and one mint circuit are required. Alternatively, a single circuit conditioned on a public **type** input subsumes all three spends; this is the more efficient deployment choice but costs a larger universal verifier.

14.1 Mint Circuit

Public inputs. **totalFace** (the total face value $\sum v_i$ of the notes being minted), $\{cm_1, \dots, cm_N\}$ (commitments to be appended), and **issuerRegistryEntry** (a Merkle root or index into IdentityRegistry attesting to the issuer’s registered credential).

Witnesses. For each note i : $(type_i, v_i, \rho_i, id_{payload,i}, predicate_i)$, plus the issuer’s private material for whatever $id_{payload}$ the note requires.

Constraints.

1. $\sum v_i = \text{totalFace}$ (amount conservation).
2. Each $cm_i = H_{cm}(type_i, v_i, \rho_i, id_{payload,i}, predicate_i)$ (opening binding).
3. For each Type 1 note: $id_{payload,i}^{(1)} = (R_{i,i}, M_{\text{issuer}} + r_{i,i} \cdot pk_{\text{issuer}})$ with $R_{i,i} = r_{i,i}G$, and M_{issuer} matches the issuer’s registered ElGamal decryption.
4. For each Type 2 note: $id_{payload,i}^{(2)} = (m_{\text{issuer}}, \sigma_i)$ with σ_i a valid Schnorr signature on cm_i under $pk_{\text{issuer}} = m_{\text{issuer}}G$.
5. For each Type 3 note: $id_{payload,i}^{(3)} = (R_{e,i}, pk_i^*)$ with $R_{e,i} = r_{e,i}G$ and $pk_i^* = H_{st}(r_{e,i} \cdot pk_{\text{recipient},i}) \cdot pk_{\text{recipient},i}$, for each note’s target recipient.

Contract side. The contract verifies the proof, pulls **totalFace** BUCK from the caller (ordinary ERC-20 transfer, caller pays their own outgoing-transfer demurrage), appends each cm_i to **noteTree**, and increments **noteFaceSum** by **totalFace**. No other state is touched – the contract’s (B, A) update through the normal transfer path.

14.2 Spend Circuit (Type 1 – Encrypted Issuer)

Public inputs. **noteRoot** (current Merkle root of **noteTree**), **nullifier**, **faceValue** (the note’s face v , revealed), **depositorRegistryEntry** (pointer into IdentityRegistry for the depositor’s credential).

Witnesses. $(v, \rho, id_{payload,i}^{(1)}, path)$, $sk_{\text{issuer}/\text{dep}}$ (shared, by Theorem 6), the depositor’s fresh encryption randomness r_d , and the depositor’s registered credential $E_{\text{dep}}^{(\text{registry})}$.

Constraints.

1. $cm = H_{cm}(1, v, \rho, id_{payload,i}^{(1)}, pred)$ opens to a leaf of **noteTree** under **path** (Merkle inclusion).
2. $nullifier = H_{nf}(1 || sk_{\text{dep}} || \rho)$.
3. $id_{payload,i}^{(1)} = (R_i, C_i)$; the registered credential $E_{\text{dep}}^{(\text{registry})} = (R_d^{\text{reg}}, C_d^{\text{reg}})$ exists in IdentityRegistry; CP-equality $C_i - sk_{\text{dep}}R_i = C_d^{\text{reg}} - sk_{\text{dep}}R_d^{\text{reg}}$ holds (Theorem 6).
4. **faceValue** = v (witness v revealed as public input).

The contract checks the nullifier is unused, reads its own (B, A) from the ERC-20 ledger, computes $\text{ageShare} = v \cdot A/B$, performs the age-preserving transfer (recipient gains v balance and ageShare BUCK-age; pool loses the same), and decrements noteFaceSum by v . No demurrage arithmetic lives in the SNARK.

14.3 Spend Circuit (Type 2 – Public Issuer)

Public inputs. noteRoot , nullifier , faceValue , issuerPkClaim (the issuer’s public-identity claim for human-readable verification, redundant with the circuit’s own check but useful for wallets), $\text{depositorRegistryEntry}$.

Witnesses. $(v, \rho, \text{id}_{\text{payload}}^{(2)}, \text{path})$, sk_{dep} , the depositor’s Schnorr nonce k_{dep} .

Constraints.

1. Merkle inclusion of cm under path (as Type 1).
2. $\text{nullifier} = H_{\text{nf}}(2 \parallel \rho)$ (publicly derivable from ρ – this is the first-come-first-served property of Type 2).
3. $\text{id}_{\text{payload}}^{(2)} = (m_{\text{issuer}}, \sigma_{\text{issuer}})$ with σ_{issuer} verifying against $pk_{\text{issuer}} = m_{\text{issuer}}G$ in IdentityRegistry.
4. Schnorr PoK of sk_{dep} for pk_{dep} in IdentityRegistry, Fiat-Shamir bound to msg.sender , chainid , nullifier .
5. $\text{faceValue} = v$.

The contract’s delivery is identical to Type 1: an age-preserving transfer of v balance and $v \cdot A/B$ BUCK-age from pool to recipient, computed outside the SNARK.

14.4 Spend Circuit (Type 3 – Stealth-Targeted)

Public inputs. noteRoot , nullifier , faceValue , $\text{depositorRegistryEntry}$. pk^* is *not* a public input: revealing it would let an observer look up $pk_{\text{recipient}}$ in IdentityRegistry and deanonymise the target. Instead, pk^* is a private witness, and the Merkle inclusion proof binds it to the commitment.

Witnesses. $(v, \rho, R_e, pk^*, \text{path})$, $sk_{\text{recipient}}$, the stealth secret $sk^* = H_{\text{st}}(sk_{\text{recipient}} \cdot R_e) \cdot sk_{\text{recipient}}$, sk_{dep} , Schnorr nonces.

Constraints.

1. Merkle inclusion of $cm = H_{\text{cm}}(3, v, \rho, (R_e, pk^*), \text{pred})$.
2. $\text{nullifier} = H_{\text{nf}}(3 \parallel sk^* \parallel \rho)$.
3. $sk^* \cdot G = pk^*$ (Theorem 7 completeness).
4. $sk^* = H_{\text{st}}(sk_{\text{recipient}} \cdot R_e) \cdot sk_{\text{recipient}}$ (Theorem 7 setup; also binds $pk_{\text{recipient}}$ indirectly via this relation).
5. Schnorr PoK of sk_{dep} for pk_{dep} in IdentityRegistry.
6. $\text{faceValue} = v$.

As with Types 1 and 2, the contract performs an age-preserving transfer of v balance and $v \cdot A/B$ BUCK-age from pool to recipient, outside the SNARK.

Optional identity-binding. If the recipient wishes to bind the deposit to their registered identity (for tax/audit purposes), the circuit additionally proves CP-equality between $E_{\text{dep}}^{(\text{registry})}$ and a re-encryption of the Identity point M decrypted from $E_{\text{recipient}}^{(\text{registry})}$, matching the recipient’s own credential. This variant is opt-in; the minimal-privacy variant simply requires *any* registered depositor.

14.5 Circuit Engineering Budget

Ballpark constraint counts (orders of magnitude, BN254 Groth16):

Circuit element	Approx. constraints
Poseidon commitment hash (1 call)	~250
Poseidon nullifier PRF (1 call)	~250
Merkle path verification (20 levels)	~5000
ElGamal / Chaum-Pedersen (Type 1)	~2000
Schnorr PoK (Types 2/3)	~500
Stealth-address relation (Type 3)	~1500
Range check on v	~100

Total spend-circuit constraint count is in the range 6,000-10,000, comparable to Zcash Sapling’s Spend circuit (~25,000 including note-commitment randomness features we don’t need) and Aztec’s basic note-spend circuit. On-chip proving time on a modern smartphone: a few seconds.

14.6 Common Circuit Gadgets

A base-level library of gadgets is shared across all spend types:

- Merkle inclusion (Poseidon path)
- Poseidon hash wrapper
- Schnorr PoK
- Range proofs on field elements (for v)

Type-specific gadgets layer on top. This is standard SNARK library architecture (Aztec Noir, Circom, Halo2 all support this layering).

15 What This Architecture Rules Out: Pure-Bearer Cryptographic Notes

The design intentionally does not provide a pure-bearer cryptographic note – one whose spend predicate is "anyone who knows the opening can deposit, with no Identity hook at all." This is the closest analogue to a \$20 bill (possession is the entirety of ownership, no registration involved), and it is attractive for the same reasons physical cash is attractive. Three properties make it a poor fit for the cryptographic surface:

1. *Issuer-race is unbounded.* A pure-bearer predicate `know(rho)` has no cryptographic mechanism preventing a malicious issuer from retaining ρ and racing the holder to deposit. The physical seal on paper makes the *observation* of ρ detectable to the holder, but does not constrain the *issuer* at mint time.
2. *Deposit has no Identity binding.* The regulatory audit path described above relies on the depositor being a registered Identity. A pure-bearer deposit with no Identity hook breaks this property and forces the system into one of two uncomfortable corners: accept unregistered deposits (losing all compliance), or reintroduce Identity at deposit time (at which point it is just a Type 2 note in different clothing).
3. *Three alternative routes all collapse into existing types.* *Blind minting* (the issuer does not know ρ at mint time) requires the holder to participate in minting with a blinded commitment – incompatible with "print a stack of notes and hand them out." *Trusted printer* (HSM-attested erasure of ρ) works in principle but reintroduces a single-point-of-trust. *Short expiry* limits the issuer-race window – equivalent to Type 2 with an additional expiry predicate.

Rather than contort the cryptographic layer to approximate pure bearer, BUCK Notes keeps the physical-world primitive where it belongs: the *tamper-evident seal over a Type 2 note* is a physical-exclusivity mechanism, and that is an inherently physical property. Cryptography makes the *opening* hard to forge; a hologram makes the *observation* detectable. The two surfaces complement without pretending to be the same primitive.

For use cases that truly require "no identity link at all":

- *Short-lived hand-to-hand circulation* is served by Type 2 paper notes with tamper-evident seals, printed by a trusted issuer who publicly warrants that they erased ρ after printing. This warranty is an off-chain social/legal claim identical in kind to the trust extended to the Bank of Canada when it issues physical polymer banknotes. A credit union or a commercial issuer fills the role of the central bank for this purpose.
- *Long-term holding* is served by Type 1 (self-issued) or Type 3 (stealth-targeted), where the cryptography alone prevents issuer-race and therefore no hologram seal is needed.

The canonical "walking-around money" use case is handled by Type 1 notes: Alice issues a stack for her own use, hands them out, and the recipients transfer the openings back to Alice via direct message for deposit at one of Alice's own accounts. This changes the UX – payment now involves a brief digital handshake – but gains a property physical cash lacks: a dropped or stolen Type 1 note is useless to anyone but Alice.

16 Security Assumptions and Theorems Used

The unified Notes design rests on the same cryptographic foundations as the existing Identity system, plus one already-proved additional theorem:

Primitive / Theorem	Usage in Notes	Assumption
PS signature (issuer credential)	Mint-circuit issuer-binding	q-SDH
ElGamal on G_1	Type 1 identity payload	DLog on G_1
Theorem 6 (CP re-encryption)	Type 1 spend predicate	DLog + ROM
Theorem 7 (stealth-targeted)	Type 3 spend predicate	DLog + ROM
Schnorr PoK (sigma protocols)	Type 2, Type 3 depositor check	DLog + ROM
Poseidon hash/PRF (SNARK-side)	Commitments, nullifiers	ROM-like (circuit)
SNARK proof system (e.g., PLONK)	Wrapping all of the above	system-specific
Merkle tree (noteTree)	Commitment anchor	collision-resistance

No *novel* cryptographic assumption is introduced. The only new primitive not already in BUCK is the Poseidon hash family, which is standard in SNARK deployments (Aztec, Zcash Sapling/Orchard, Starknet, Mina) and well-studied.

16.1 Demurrage Conservation as a Contract Invariant

Because the pool is a regular BUCK account and redemption uses a two-sided-balanced age-preserving transfer, no Notes-specific demurrage invariant is needed. The system-wide BUCK invariants

$$\sum_{\text{all accounts}} B_X + B_{\text{Jubilee}} = \text{const}, \quad \frac{d}{dt} \sum_{\text{all accounts}} A_X = \sum_{\text{all accounts}} B_X$$

hold trivially through redemption: the v balance and $v \cdot A/B$ age moved from pool to recipient appear on both sides of each sum. The Notes contract only needs to maintain two auxiliary local invariants:

1. $F \geq 0$ (no negative note face sum).
2. Every v appearing in a deposit equals the v that appeared in the corresponding mint (enforced by the SNARK via the Merkle inclusion of cm binding v).

Both are trivial. The only new mechanism is the age-preserving transfer primitive, itself a simple state-machine operation on two accounts' (B, A) pairs.

17 Implementation Scope

Rough engineering estimate, assuming the existing Alberta Buck contract skeleton and an existing SNARK toolchain (Aztec Noir or Circom/snarkJS):

17.1 Smart Contract Additions

- `noteTree` (Merkle tree with Poseidon nodes): ~1 week, bulk of code is existing Aztec/Zcash sapling implementations.
- `nullifiers` mapping: trivial.
- `noteFaceSum` scalar + mint/deposit updates: trivial (audit only).
- Age-preserving transfer primitive (system-level hook writing to two accounts' (B, A) atomically): ~1 week.
- Three spend-circuit verifiers (Groth16-style on-chain): ~1 week integration per circuit, bulk is circuit-side.
- Mint-circuit verifier: ~1 week.

Total: ~7 weeks for an experienced Solidity engineer.

17.2 SNARK Circuits

- Mint circuit: ~2-3 weeks (Poseidon commitments, ElGamal encryption, Schnorr signature verification inside the circuit).
- Spend circuits (three, with shared gadgets): ~4-6 weeks.
- Common gadgets library (Merkle, Poseidon wrappers, Schnorr, range checks): ~2 weeks.

Total: ~10 weeks for an experienced SNARK engineer.

17.3 Wallet & UX

- Note scanning / recognition for Type 3 (stealth-address scan): ~1 week.
- Type 1 self-mint-to-private-address flow: ~1 week.
- Type 2 cheque creation and deposit UX: ~2 weeks.
- Type 3 targeted-note UX: ~2 weeks.
- QR-print templates, hologram-seal vendor integration: ~1 week.
- Prover service integration for thin clients: ~2-3 weeks.

Total: ~9 weeks for a wallet team.

17.4 Cross-Checks and Audit

- Cryptographic review of the spend circuits: 4 weeks, external auditor with SNARK-circuit expertise.
- Economic model validation (pool-rate demurrage in stress scenarios: low turnover, high turnover, mint/deposit mismatch, large accumulated lost-note face sum): 2 weeks.
- End-to-end fuzz and property-based testing: 2 weeks.

Total: ~8 weeks of external + internal audit.

17.5 Phased Rollout

The engineering work is naturally phaseable:

Phase	Scope	Dependencies
1	On-chain note pool: <code>noteTree</code> , nullifiers, mint-verifier	Existing BUCK ERC-20 deployment
2	Type 1 spend circuit + chain-only self-wormhole wallet UX	Phase 1, Identity CP gadget
3	Paper-note QR export/import, printer-template library	Phase 2, hologram vendor
4	Type 2 spend circuit + signed-cheque wallet UX	Phase 1
5	Type 3 spend circuit + stealth-address scan UX	Phase 2, Proofs Theorem 7 gadget
6	Programmable <code>predicate</code> field (bounded predicate DSL)	Phases 1-5, Aztec Noir toolchain
7	Optional: off-chain re-targeting via proxy re-encryption	Phases 1-5, Identity Guardians

Phases 1-2 establish the minimum viable Notes system (self-wormhole only). Phases 3-5 ship the full use-case matrix (paper notes, cheques, targeted gifts). Phases 6-7 are optional extensions deferred until the core system is production-proven.

Overall program estimate: 6-9 months of focused work to ship Phases 1-5 as a production-quality BUCK Notes system. Phases 6-7 add 4-8 months of further engineering if demand materialises.

17.6 Regulatory and Legal Engagement

In parallel with engineering, the legal and regulatory work:

- Confirm that BUCK Notes falls within Alberta’s provincial jurisdiction over property and civil rights (s 92(13)) – the analysis in the Legal Foundation document covers the underlying primitive; bearer notes are an extension that may warrant additional treatment.
- Engage FINTRAC on the issuance-side reporting model.
- Engage the Office of the Superintendent of Financial Institutions (OSFI) on reserve treatment.
- Coordinate with Alberta Treasury Branches (ATB) as a candidate first issuer of paper notes under a public-warranty of ρ -erasure for the Type 2 walking-around-cash surface.

18 Open Questions

The architecture above leaves a handful of economic and cryptographic questions worth sharpening before commitment:

18.1 Donation Incidence

Donations to the contract lower A/B and therefore lower the age-share every subsequent redeemer absorbs. The benefit spreads across all future redemptions rather than accruing to a single lucky first-redeemer. Because the age-share calculation uses the *current* A/B at each redemption, a donation immediately before a large redemption benefits that redemption more than a small one – but every subsequent redemption benefits in proportion. This is reasonable behaviour without any special smoothing logic.

Residual donation principal (after all notes drain) is not lost: it remains in B with $A = 0$, available to subsidise the next generation of mints (whose recipients will absorb an artificially low A/B at their redemption).

18.2 Stranded Value on Lost Notes

A lost note contributes to F indefinitely. The BUCK backing that note sits in the pool, accumulating BUCK-age. Since the age-preserving transfer uses A/B (not A/F), the lost face value weighs on A through continued age accumulation, and B remains unclaimed. Redemptions of live notes absorb an ever-larger A/B as the lost share’s age dominates. At 1%/yr loss and 2%/yr demurrage, the long-term tendency is that the lost note’s implicit demurrage is paid instalment-style by subsequent live noteholders.

A reclaim path is possible: after a very long period (say 99 years), the contract could allow the issuer to submit a zero-knowledge proof that a specific note commitment remains unspent, and reclaim the residual principal less accrued demurrage. Requires issuer-side recordkeeping; deferred.

18.3 Integer-Arithmetic Rounding

Redemption involves two integer divisions (computing $v \cdot A/B$ for the age-share). Any residual from truncated division remains in the pool’s (B, A) state, raising A/B slightly for the next redeemer – the rounding is self-healing rather than drift-accumulating. Choice of rounding direction (floor vs. banker’s) is a minor contract-level decision with no cryptographic impact.

18.4 Predicate Programmability vs. Circuit Simplicity

Keeping three fixed types simplifies the circuit count. Adding programmable predicates (time locks, oracle conditions, multi-sigs) re-introduces most of the complexity that the fixed type scheme was chosen to avoid. A compromise: support a *bounded* predicate language in the circuit (time-lock + oracle-condition + up-to-3-sig) rather than a fully general one. The bounded language covers 90%+ of the programmable-predicate use cases while keeping the verifier compact.

18.5 Universal Spend Circuit vs. Three

Deploying three distinct spend verifiers costs gas per transaction (one verifier per call). A universal verifier that takes `type` as a public input and branches internally costs a larger but single verifier. Gas trade-off depends on production transaction mix; if Type 1 dominates (Wormhole-style), three verifiers are cheaper. Empirical question, resolvable at deployment time.

18.6 Regulatory Interaction with the Age-Preserving Transfer

Jurisdictional regulators (FINTRAC, OSFI) have rules around bearer instruments and tax treatment of negative-interest accounts. BUCK's on-chain demurrage was argued through separately, and the Notes contract inherits that treatment with no pool-specific novelty – the contract is just another BUCK account.

The one new wrinkle is the *age-preserving transfer* itself: ordinary BUCK transfers carry only balance (recipient gets "fresh" BUCK), whereas Note redemptions carry balance plus a proportional BUCK-age. This means a Note recipient's post-redemption spendable value is less than their nominal balance – the difference being the demurrage they will pay on their next outgoing transfer. For tax and reporting purposes, whether the "face" or the "spendable" number should be used as the income figure is a question for tax counsel. The on-chain state is fully transparent either way.

DRAFT

19 Summary

BUCK Notes is the Alberta Buck's native private-bearer primitive. A single Merkle tree of commitments, a single nullifier set, and three Identity-binding types – Encrypted-Issuer (Type 1, self-wormhole), Public-Issuer (Type 2, signed cheque), Stealth-Targeted (Type 3, addressed cheque) – span every use case the Alberta Buck series needs a bearer instrument to cover, from cold-storage cryptographic self-wormholes to walking-around paper notes to cross-border targeted remittances.

Demurrage needs no note-side machinery: the Note Pool is literally the BUCK ERC-20 contract's own balance – a regular BUCK account with its own B and A in the ERC-20 ledger. The only Note-side state beyond the Merkle tree and nullifier set is one scalar F , the total face value of live notes, kept as an audit quantity. Redemption delivers the *full face value* v to the recipient, together with the pool's proportional share of BUCK-age $v \cdot A/B$; the recipient absorbs this age onto their own account and bears the demurrage cost on their next outgoing transfer through the ordinary BUCK accounting path. This is exactly the demurrage they would have paid had they held v BUCK personally through the pool's residence period. The entire mechanism requires a single new primitive, an age-preserving transfer from pool to recipient, localized to the Notes contract's one egress function. Donations to the pool dilute A/B and cheaply subsidise all subsequent redeemers. Conservation of balance, age, and Jubilee growth all fall out exactly.

The printer-trust distinction produces a sharp partition of which note types are safe for long-term holding: only *Type 1* (issuer = beneficiary) and *Type 3* (stealth-targeted to beneficiary) can be held for arbitrary durations. *Type 2* is cryptographically a short-term instrument regardless of how long a user might wish to keep it, because the issuer retains the spending secret. This is an intrinsic property that cannot be remedied by additional cryptography; it follows directly from the Type 2 identity payload carrying the issuer's public identity and no recipient-binding.

The cryptographic architecture does not attempt to provide a pure-bearer note. That use case remains the domain of physical trust models: tamper-evident seals over Type 2 paper notes, issued by a trusted printer with a public warranty that the opening was erased after printing. This keeps the cryptographic/physical boundary explicit rather than contorting the cryptographic layer to approximate a property that inherently requires physical primitives.

A BUCK Note can live on chain as a commitment-opening pair, travel between wallets as a digital message, or be printed on paper as a QR-encoded slip with a hologram seal (for Type 2) or without one (for Types 1 and 3). All three surfaces share the same cryptography and the same on-chain footprint. Counterfeiting is cryptographically impossible: a "fake" note's commitment is simply absent from the on-chain tree, the Merkle proof fails, and the wallet displays **FAKE**. Lost notes are not extracted as seigniorage: the pool-age mechanism returns their value to the commons via demurrage pressure on subsequent live-note redemptions.

Issuance is on chain and identity-bound. Circulation is private. Deposit is identity-bound and auditable. Selective-disclosure audit paths give regulators access to the issuance trail without observing day-to-day circulation. This is stronger compliance than physical cash provides today, and stronger privacy than any CBDC has ever proposed.

No novel cryptographic assumption is introduced. Theorems 6 and 7 of the Proofs document (Proofs) apply verbatim to the Type 1 and Type 3 spend predicates, respectively; Type 2 reduces to Schnorr. The only new circuit-level primitive is Poseidon, which is already standard across the SNARK ecosystem. Implementation scope for Phases 1-5 (everything except programmable predicates and off-chain re-targeting) is 6-9 months for a focused team.

BUCK Notes answers the question Chaum was asking in 1982: what does digital cash look like when you take "cash" seriously? The answer, forty years later: it looks like a commitment in an on-chain pool, openable as a message, a QR, or a printed slip, with a value that decays at the same

rate your bank balance does, integrated with a complete monetary system that does not require a central bank to function. Alberta has, uniquely in the world, all four pieces required to build this. Whether to assemble them is, as ever, the only question that matters.

DRAFT